

Schema-Graph-Constrained Natural Language Interface for Relational Materials Databases:

Schema Graph 制約型自然言語インターフェースによる正規化材料データベースへのアクセス

Satoshi Minamoto^{a,*}

^aNational Institute for Materials Science (NIMS), 1-2-1 Sengen, Tsukuba, Ibaraki 305-0047, Japan

Abstract

正規化リレーショナルデータベース (RDB) に格納された材料データへのアクセスには SQL 知識が必要であり、材料研究者にとって大きな障壁となっている。本研究では、自然言語による参照系クエリを安全な SQL 文へ自動変換する **Schema Graph 制約型 Text-to-SQL (T2SQL)** 手法を提案する。外部キー (FK) 関係をグラフとして表現し、Steiner 木近似で最小 JOIN パスを算出する **Schema Graph 走査エンジン** を中核に、日英バイリンガル材料用語辞書、Few-Shot RAG、8 層 SQL ガード、Coverage Score、Intent Classifier の 6 要素を統合する。

OQMD (~30 テーブル) の実スキーマ規模を想定した **30 テーブル・31FK 関係・150 クエリ** の検証を実施した。6 カテゴリ (Simple/Medium/Complex/Very Complex/Cross-domain/Aggregation) 各 25 件の 150 クエリで評価した結果：走査なし (Full Schema 30 テーブル全量注入) 成功率 **86.7%** に対し、走査あり (Traversed: 関連テーブルのみ提示) **94.7%** (+8.0pp)、不要 JOIN 率 9.2%→2.8%に改善。Medium カテゴリでは+16pp (76%→92%) の改善を示した。Naive Rule-based (全テーブル JOIN) は 30 テーブルで実行成功率 **0%** に破綻し、Schema Graph 走査 + Rule-based は 54%、LLM + 走査が最良であった。McNemar 検定により走査効果の統計的有意性を確認した ($p = 0.031$)。

Core サブセット (7 テーブル・OQMD 1,351 件) でのアブレーション実験も実施した。gpt-5.5 は 7 テーブルで天井効果 (全条件 100%) を示し、Schema Graph 制約のみで Few-Shot RAG の追加効果が消失する。一方 gpt-4o-mini では RAG により 80%→98% (+18pp) と顕著な改善が確認され、Schema Graph 走査がモデル世代に依存しない構造的改善であることを実証した。

Rule-based モードは 32–60 ms のレイテンシを実現し、LLM モードは 2.4–6.1 秒 (49–194 倍遅延) だが辞書外語彙に対応する。Intent Classifier により out-of-scope 正解率を 0%→100%に改善した。本手法は実行時 FK イントロスペクションに基づくため、Materials Project、AFLOW、機関内 DB 等への展開が可能であり、AI4S 駆動型材料探索ワークフローの基盤的構成要素を提供する。

Keywords: Text-to-SQL, 材料データベース, スキーマグラフ, 検索拡張生成 (RAG), OQMD, 金属間化合物, 自然言語処理, マテリアルズ・インフォマティクス, AI for Science

1. 緒言

1.1. AI for Science と材料インフォマティクスにおけるデータアクセス障壁

材料データベースにおける自然言語インターフェースの本質的課題は、SQL 文の構文生成そのものではなく、正規化された異種材料データを、外部キー関係と材料ドメイン意味論に従って正しく結合することである。本論文では、この課題に対し Schema Graph 走査 (FK 関係のグラフ表現と Steiner 木近似) に基づく制約型手法を提案する。30 テーブル・31FK 関係・150 クエリの実運用規模検証において、走査なし成功率 86.7% に対し走査あり **94.7%** (+8.0pp)、不要 JOIN 率 9.2%→2.8%の改善を実証し、Naive Rule-based は 30 テーブルで実行成功率 0% に破綻することを示す。

AI for Science (AI4S) —科学的発見を加速するための人工知能の体系的応用—は、材料研究を変革しつつある [1]。AI4S は、AI エージェントが自律的に仮説を生成し、大規模データベースから関連データを検索し、実験やシミュレーションを設計し、結果を解釈する統合的ワークフローを構想している。このビジョンにおいて、構造化された材料データへの自然言語アクセスは単なる利便性ではなく前提条件である：AI エージェント (またはそれを指揮する研究者) が正規化リレーショナルデータベースを効率的にクエリできなければ、AI4S パイプライン全体がデータ検索段階でボトルネックとなる。

ハイスループット第一原理計算の急速な発展により、複数の大規模材料データベースが構築されている：Open Quantum Materials Database (OQMD、約 100 万件) [2, 3]、Materials Project (15 万件超) [4]、AFLOW (350 万件超) [5]、NOMAD [6]。これらのデータベースは計算材料探索、候補相のスクリーニング [7, 8]、実験観測の検証に不可欠であ

*Corresponding author

Email address: minamoto.satoshi@nims.go.jp (Satoshi Minamoto)

る。一方、材料データの構造化・知識化の取り組みとして、Ishii らはレガシー超伝導データベース SuperCon のオントロジー構築と RDF 化 [9] や、高分子データベース PoLyInfo の機械可読化 [10] を実現しており、スキーマの意味的構造化が材料データ活用 の前提条件であることを示している。

しかし、正規化リレーショナルスキーマ（外部キーで接続された複数テーブル）に格納されたデータへのアクセスには SQL 知識が必要である。材料工学において本質的に重要なのは、大量データの収集よりも少量の異種データを正確に組み合わせることである：組成・結晶構造・熱力学的安定性・計算条件・物性値はそれぞれ独立したテーブルに正規化されており、これらを外部キー（FK）に沿って正しく JOIN することで初めて意味のある材料情報が得られる。例えば、「Fe を含む安定な B2 化合物の格子定数」を検索するには、元素・プロトタイプ・熱力学的安定性に対する WHERE 条件を持つ 4 テーブル JOIN が必要となる—データベース技術者にとっては容易だが、材料研究者にとっては非自明な操作である。不正確な JOIN（不要なテーブル結合や結合条件の欠落）は誤ったデータの組み合わせを返し、材料探索やスクリーニングの判断を誤導する深刻なリスクとなる。

多くの材料データベースが提供する REST API や GraphQL エンドポイントはこの障壁を部分的に緩和するが、RDB 全般に共通する根本的制約が残る：(a) 事前定義されたフィルタ構文への依存（アドホックな複合条件や集約が困難）、(b) スキーマ更新時の API 非互換リスク、(c) 機関内データベースや独自拡張スキーマには API が存在しない場合が多いこと。T2SQL アプローチは、スキーマ構造を実行時に解析するため、API の有無に関わらず任意の正規化 RDB に対して原理的に展開可能である。

AI4S の文脈において、このデータアクセス障壁は特に深刻である：材料スクリーニングや物性予測を行う自律 AI エージェントは、複雑な複合条件データベースクエリをプログラマ的に発行する必要がある。正しい SQL を確実に生成する自然言語インターフェースは、人間の研究者と AI エージェントの双方がデータベース工学の専門知識なしに材料データベースと対話することを可能にし、AI4S ワークフローの重要なボトルネックを除去する。

1.2. Text-to-SQL の技術動向

Text-to-SQL (T2SQL) 研究は大規模ベンチマークにより急速に進展している：Spider [11] (10,181 問、200 データベース、138 ドメイン)、BIRD [12] (12,751 問、95 データベース、33.4 GB)。最近の LLM ベース手法は高い精度を達成している：DAIL-SQL [13] はスケルトンベースの few-shot 事例選択により Spider 上で実行精度 86.6% を報告し、DIN-SQL [14] は分解型文脈内学習により 85.3% を達成している。Hong et al. [15] はスキーマリンキングの支配的役割を指摘する包括的サーベイを提供し、Maamari et al. [16] は高度な LLM により明示的スキーマリンキングが不要になる可能性を論じているが、これは議論が続いている。

しかし、既存の **T2SQL** ベンチマークはすべて汎用ドメイン（e コマース、医療、教育）を対象としており、材料科学データベース固有の課題に取り組んだ先行研究は存在しない：ドメイン固有の用語（Strukturbericht 記号、プロトタイプ名）、バイリンガルクエリ（日英元素名）、組成テーブル正規化（化合物ごとに元素 1 行）、熱力学的安定性フィルタ ($E_{\text{hull}} \leq 0.001$ eV/atom) などの課題である。

1.3. 材料インフォマティクスにおける NLP

材料科学における NLP 応用は拡大している：MatSciNLP [17] は材料テキストの 7 つの NLP タスクをベンチマークし、LLAMAT [18] は LLaMA の継続事前学習により情報抽出を改善し、Materials Knowledge Navigation Agent (MKNA) [19] は自然言語の意図をデータベース検索・構造生成アクションに変換し、OptiMat Alloys [20] は LLM 駆動型対話式合金探索ツールを提供している。Materials Project は MCP ベースの LLM 統合の提供を開始している [21]。

これらの研究は情報抽出やエージェントベースのデータベース対話を扱っているが、正規化リレーショナルデータベース上のスキーマ認識型 **SQL** 生成—特に外部キーグラフ走査による自動 JOIN 経路探索—を対象としたものはない。このギャップが本手法の焦点である。

1.4. オントロジー・知識グラフアプローチとの関連

材料データへの構造化アクセスには、オントロジー・知識グラフ (KG) に基づくアプローチも有力な系譜を形成している。Ishii らは超伝導材料データベース SuperCon を RDF オントロジーとして再構築し、SPARQL エンドポイントを通じた意味検索を実現した [9]。同グループは PoLyInfo の高分子データについても同様の機械可読化を行い、BFO 上位オントロジーとの整合により分野横断的なデータ連携を可能にしている [10]。欧州では Elementary Multiperspective Material Ontology (EMMO) が材料モデリングの標準オントロジーとして整備され、OWL 推論に基づく意味的一貫性の保証を提供している [22]。

汎用ドメインでは、Sequeda & Allemang [23] が企業 SQL データベースを知識グラフに変換し、Text-to-SPARQL が Text-to-SQL に比べ正解率を 16% から 54% に向上させることを示した。さらにオントロジーの意味制約を用いたクエリ検証 (OBQC) により 72% まで改善している [24]。

本手法はこれらオントロジーアプローチと構造制約によるクエリ品質保証という目的を共有する。オントロジーの OWL プロパティにおける domain/range 制約は、RDB における FK 関係のテーブル間制約と機能的に対応し、OBQC による SPARQL クエリ検証は、本手法の SQLGuard (カラムホワイトリスト・FK 整合 JOIN 検証) と同等の役割を果たす。ただし、両アプローチの適用文脈は異なる。オントロジーアプローチは RDF トリプルストアへのデータ変換を前提とし、OWL 推論による意味的推移律（例：「NiAl は L1₂ 型」→「NiAl は金属間化合物」）や異種データベース間の相互運用性において優れた表現力を持つ。一方、OQMD・Materials Project・AFLOW 等の大規模材料データベースは既にリレーショナルスキーマで運用されており、本手法はこれらの既存 **RDB** インフラをそのまま活用する設計を採用している。FK 関係のグラフ走査 (Steiner 木近似) は、オントロジーの推論に依存せずに JOIN パスの正確性を保証する軽量な代替手段を提供する。

両アプローチは排他的ではなく相補的である。将来的には、オントロジーが定義する意味的制約を Schema Graph の走査ヒューリスティックに統合することで、FK 関係だけでなくドメイン知識に基づく JOIN パス選択が可能になると考えられる。

1.5. 目的と新規性

本研究の貢献は以下の5点である：

1. **Schema Graph 制約型 T2SQL 手法**：日英バイリンガル材料用語辞書と FK 関係グラフ走査を組み合わせ、正規化材料データベース上の自動 JOIN 経路計算を実現する。材料工学で本質的に重要な「少量の異種データの正確な組み合わせ」を、Steiner 木近似による最小 JOIN パス算出で保証する。Schema Graph 制約は、結果正解率を維持しつつ不要 JOIN を排除し、Rule-based でも LLM なしに高精度を達成する主要因である。
2. **8 層安全パイプライン**：Coverage Score（未認識語彙検出）、Intent Classifier（スコープ外質問検出）、数値条件パーサー、化学式パーサー、多層 SQL ガード（カラム WL・JOIN 検証）を統合し、Silent Constraint Dropping（未登録語彙の無通知破棄）を防止する。
3. **RAG アブレーション研究**：4 条件(no examples / manual / paper / all) × 50 クエリの比較で、gpt-5.5 では Schema Graph 制約のみで天井効果が生じ、Few-Shot 事例の追加効果が消失する条件を定量的に示す。
4. **100 クエリ VASP フォーラムストレステスト**：SQL-answerable、数値条件、曖昧、スコープ外、安全性の5カテゴリにわたる現実的な材料研究者クエリを模擬した大規模評価。
5. **カスケード Rule-based → LLM アーキテクチャの定量評価**：Rule-based 32–60 ms vs gpt-5.5 2.4–6.1 秒 (49–194 倍) のレイテンシ・正確性トレードオフを実測。
図 1 に本手法の解析フロー全体を示す。

2. 材料 Text-to-SQL の設計要件

材料データベースに対する自然言語インターフェースの本質的課題は、SQL 文の構文生成そのものではなく、正規化された異種材料データを、外部キー関係と材料ドメイン意味論に従って正しく結合することである。汎用 Text-to-SQL システムを材料データベースに直接適用した場合、以下の材料固有の要件が満たされず、重大な精度低下を招く。

1. 正規化組成テーブルの正確な **JOIN**：多元素化合物の各元素が独立行として格納されるため、「Ni と Al を両方含む」検索には SELF-JOIN または INTERSECT が必要となる。汎用 LLM はこのパターンを頻繁に誤る。
2. 材料用語の曖昧性解消：「NiAl」が特定化合物 (B2-NiAl) か含有元素条件かは文脈依存であり、「安定」が $E_{\text{hull}} \leq 0.001$ eV/atom か $\Delta G_f < 0$ かはスキーマ依存である。ドメイン辞書なしではカラムマッピングが不定となる。
3. プロトタイプ・空間群の語彙正規化：L1₂、AuCu₃ 型、Cu₃Au 型は同一プロトタイプを指すが、表記が多様であり、日英バイリンガルでの同義語展開が必要。
4. **FK** に沿った最小 **JOIN** パスの計算：30 テーブル規模のスキーマでは候補 JOIN パスが爆発し、不要テーブルの結合が LLM 精度を低下させる。Steiner 木近似による最小サブグラフ抽出が不可欠。
5. 数値条件の単位・閾値解釈：「バンドギャップが大きい」→ > 1.0 eV か > 2.0 eV か、「安定な」→ $E_{\text{hull}} \leq 0.001$ か ≤ 0.1 かは材料ドメイン知識に基づく判断が必要。
6. スコープ外質問の拒否：VASP 計算パラメータの最適化や DFT ワークフローに関する質問は SQL-answerable でなく、無理に SQL を生成すると無意味な結果を返す。

7. 安全実行の保証：SELECT 以外の SQL (INSERT/UPDATE/DELETE) の排除、無制限結果セットの防止（自動 LIMIT 注入）、テーブル・カラムホワイトリストによるインジェクション防止。

これらの要件は、汎用 Text-to-SQL ベンチマーク (Spider、WikiSQL 等) では評価対象に含まれない材料データベース固有の課題である。本手法は上記 7 要件すべてに対応する統合フレームワークとして設計された。

3. 手法

本節では各技術要素を詳述する。システム全体のパイプラインアーキテクチャを図 2 に示す。

3.1. 材料データのためのデータベーススキーマ設計

3.1.1. OQMD 実規模スキーマの設計 (30 テーブル・31FK 関係)

OQMD は内部的に Django ORM + PostgreSQL で ~30 テーブルの正規化 RDB として運用されており、材料データベースに典型的なスキーマ複雑性（多対多関係、正規化された組成テーブル、構造パラメータの分離）を備えた好例である。本研究では、この OQMD 実規模を想定した 30 テーブル・31FK 関係のリレーショナルスキーマを設計した (図 3)。設計は Boyce-Codd 正規形 (BCNF) に従い、以下の材料ドメイン固有の考慮に基づく。

組成テーブルの分離. 単一の化合物が複数の元素を含む場合がある (例: $\text{Fe}_3\text{Al} \rightarrow \text{Fe}$ と Al)。各元素を composition テーブルの個別行として格納することで、文字列解析なしに柔軟な元素ベースクエリ (AND, OR, NOT) を可能にする。ただし、この正規化は重要な複雑性を導入する：「Ni と Al の両方を含む化合物」のクエリは単純な `WHERE element = 'Ni' AND element = 'Al'` では不可能である（単一行が両条件を同時に満たすことはないため 0 行が返る）。代わりに EXISTS 副問い合わせが必要となる (3.3.4 節参照)。なお、この設計により化学式の表記順序 (例: Ni_3Al と AlNi_3 は同一化合物だがデータベースによって表記慣例が異なる) に依存しない検索が実現される。元素と原子分率を個別行で格納するため、式全体の文字列比較ではなく元素単位のカエリで同一化合物を一意に特定できる。

構造/相安定性/計算の分離. 結晶構造情報 (プロトタイプ、格子定数)、熱力学的安定性 (hull 上エネルギー)、計算メタデータ (DFT 汎関数) を別テーブルに格納し、独立した更新・拡張を可能にする。

計算物性の *Entity-Attribute-Value* (EAV) パターン. `calculated_property` テーブルは EAV パターン (`property_name`, `value`, `unit`) を使用し、スキーマ変更なしに任意の計算物性 (体積弾性率、せん断弾性率等) を収容する。

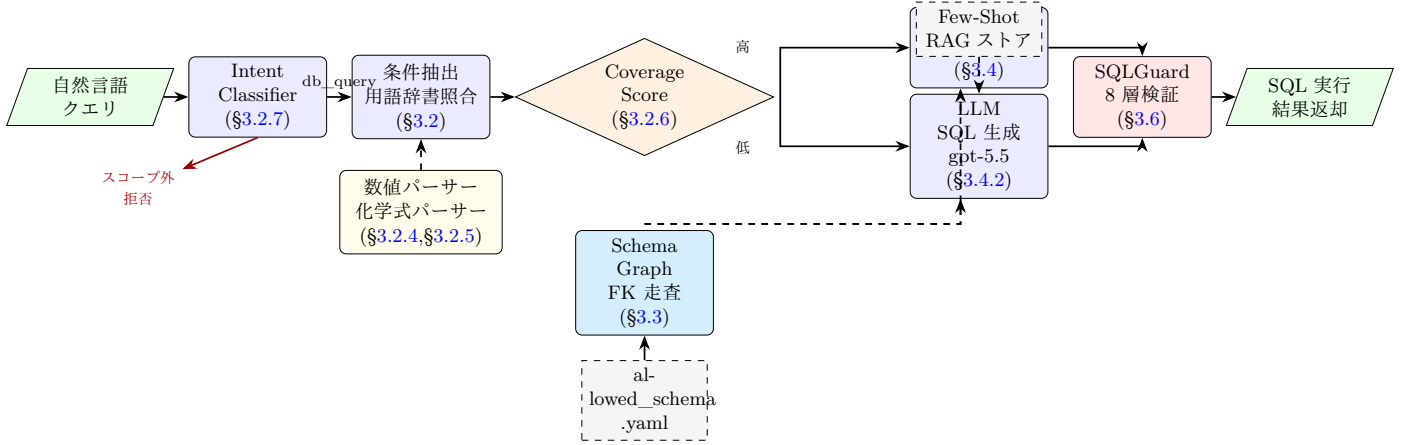


Figure 1: 提案手法の解析フロー全体図。自然言語クエリは Intent Classifier でスコープ判定後、用語辞書・数値/化学式パーサーで条件抽出される。Coverage Score に基づき Rule-based (高認識率) または LLM (低認識率) 経路に分岐し、いずれも Schema Graph (FK 走査) による JOIN パス制約と SQLGuard 8 層検証を経て安全な SQL が生成・実行される。破線矢印はデータ参照を示す。

Table 1: Core サブセットスキーマ概要 (7 テーブル)。30 テーブルスキーマの中核部分として機能する。

テーブル名	主キー	外部キー	主要カラム
material_entry	entry_id	—	formula, reduced_formula, chemical_system
composition	composition_id	entry_id	element, atomic_fraction
structure	structure_id	entry_id	prototype, strukturbericht, lattice_a, space_group
phase_stability	stability_id	entry_id	formation_energy_per_atom, energy_above_hull, band_gap
calculation	calculation_id	entry_id	method, functional
calculated_property	property_id	calculation_id	property_name, value, unit
prototype_definition	prototype_id	—	prototype_name, strukturbericht, description

Fig. 1: System Architecture — Schema-Graph-Assisted Text-to-SQL Pipeline

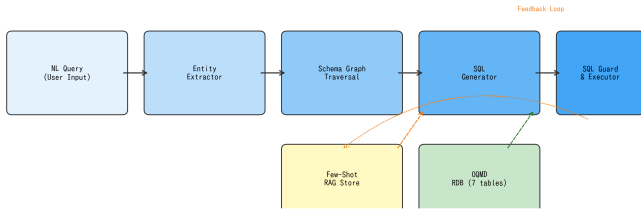


Figure 2: Schema Graph 支援 Text-to-SQL パイプラインのシステムアーキテクチャ。実線矢印は主要データフロー、破線矢印は Few-Shot RAG フィードバックループ (成功したクエリペアが蓄積・検索される) を示す。SQL ガードはデータベース実行前の最終安全層として機能する。

ER 設計の汎用性に関する注記。OQMD は内部的にリレーショナルデータベース (Django ORM + PostgreSQL) で運用されているが、本研究では API から取得したフラット JSON データに対し、NetworkX グラフおよび YAML スキーマとして走査しやすい ER 図を新規に設計した。この設計判断により、本手法の適用可能性は以下の二つのシナリオに一般化される: (1) 既存 RDB のスキーマ情報 (FK 制約) をそのまま抽出して走査する場合、(2) フラットデータから目的に応じた ER 図を新規構築して走査する場合。いずれの場合も、FK 関係がグラフとして表現可能であれば Schema Graph 走査による JOIN パス最適化が適用可能である。

3.1.2. 30 テーブルスキーマのグループ構成

30 テーブルスキーマは 6 グループから構成される。Core (7 テーブル、表 1) を中核に、以下の 5 グループ 23 テーブルを配置する: (1) **Application** (application_domain, use_case, alloy_system 等): 材料の応用先とユースケースを管理、(2) **Experiment** (experiment, measurement, sample 等): 実験データとサンプル管理、(3) **Electronic** (band_structure, dos_data, dielectric_property 等): 電子構造・誘電特性、(4) **Thermo** (thermal_property, phonon_data, thermal_expansion 等): 熱物性・フォノンデータ、(5) **Alloy** (heating_composition, alloy_phase, mixing_enthalpy 等): 高エントロピー合金と合金相管理。material_entry が中央ハブとして 21 本の FK 参照を受け入れ、application_domain は親子関係の自己参照 FK を含む。スキーマ深度は最大 3 (entry→calculation→calculated_property) であり、実際の材料 DB に典型的な FK 構造を再現している。

3.1.3. RAG コンテキスト注入のための XML/YAML 表現

スキーマ構造は YAML (allowed_schema.yaml) としてシリアル化され、二重の目的に供される: (a) SQL ガードのテーブル/カラムホワイトリスト (3.6 節)、(b) LLM プロンプトへの RAG コンテキストスキーマ YAML がシステムメッセージに注入され、LLM が利用可能なテーブル・カラム・型を理解する。このアプローチは汎用 T2SQL のスキーマシリアル化技法 [13] と類似するが、材料データベースのドメイン固有カラム意味論 (例: energy_above_hull は E_{hull} を表す) に特化している。



Figure 3: 30 テーブルスキーマの Entity-Relationship 図。material_entry（中央ハブ、21 本の FK 受入）を中心に放射状配置。6 グループ（Core/Ap-plication/Experiment/Electronic/Thermo/Alloy）を色分け表示。31FK 関係、最大深度 3。

3.2. 材料用語辞書を備えた条件抽出器

条件抽出器は、YAML ベースの日英バイリンガル用語辞書 (`material_terms.yaml`) を用いて、自然言語クエリを構造化された条件辞書に変換する。

3.2.1. 辞書構造

辞書は 4 つのカテゴリを含む：

1. プロトタイプ別名：プロトタイプ名とその変種の対応。
例：B2 ↔ [B2, CsCl, CsCl 型, B2 型]
L12 ↔ [L1₂, L12, AuCu₃, Cu₃Au 型, γ']
2. 元素マッピング：元素記号 ↔ 日英名対応。
例：Fe ↔ [Fe, 鉄, iron]
3. 安定性用語：自然言語 → 数値閾値。
「安定」/ “stable” → `energy_above_hull ≤ 0.001 eV/atom`
「準安定」/ “metastable” → `energy_above_hull ≤ 0.05 eV/atom`
4. 物性用語：物性記述からテーブル. カラム参照への対応。
「格子定数」/ “lattice constant” → `structure.lattice_a`
「形成エネルギー」/ “formation energy” → `phase_stability.formation_energy_per_atom`

3.2.2. Unicode 正規化とパターンマッチング

Unicode 下付き文字 (𐀀𐀁𐀂𐀃𐀄𐀅𐀆𐀇𐀈𐀉) はマッチング前に ASCII 数字に正規化される。元素記号認識はワード境界正規表現を使用し、独立した「Fe」を「FeCoNi」部分文字列から正しく区別する。

3.2.3. 出力形式

抽出器は構造化辞書を返す：

Listing 1: 条件抽出の例。

```
1 # 入力: "Feを含む安定なB2化合物を出して"
2 # 出力:
3 {
4   "prototype": "B2",
5   "contains_elements": ["Fe"],
6   "stability": "stable",
7   "sort_by": None,
8   "sort_order": None
9 }
```

3.2.4. 数値条件パーサー

条件抽出器は、自然言語中の数値比較表現を構造化された WHERE 句条件に変換する正規表現ベースの数値条件パーサーを含む。対応する 5 種類のパターンを表 2 に示す。

Table 2: 数値条件パーサーの対応パターン。

パターン種別	入力例	出力
記号比較	<code>band_gap > 1.0 eV</code>	<code>WHERE band_gap > 1.0</code>
日本語後置	形成エネルギーが 0.5 以上	<code>WHERE ... >= 0.5</code>
日本語より	格子定数が 3.0 より大きい	<code>WHERE lattice_a > 3.0</code>
符号判定	形成エネルギーが負	<code>WHERE ... < 0</code>
範囲指定	band gap 0.5–2.0 eV	<code>WHERE ... BETWEEN 0.5 AND 2.0</code>

物性名から対応カラムへのマッピングは内部辞書 (`_PROPERTY_COLUMN_MAP`) により管理され、「格子

定数」→ `structure.lattice_a`、「形成エネルギー」→ `phase_stability.formation_energy_per_atom` 等の変換を行う。単位変換 (eV, Å, GPa 等) も内蔵するが、現スキーマでは OQMD の内部単位がそのまま格納されているため、変換係数は全て 1.0 である。

3.2.5. 化学式パーサーと曖昧性処理

化学式パーサーは、クエリ中の化学式トークン (例: Ni₃Al, NiAl, Fe₂CoAl) を検出し、以下の 3 種類の解釈に分類する：

- **exact_formula**：化学量論比が明示された場合 (例：Ni₃Al → `formula = 'Ni3Al'`)。組成テーブルの化学式カラムとの完全一致検索を生成する。
- **formula_or_reduced_formula**：化学量論比が省略されているが、化学式単体として使用されている場合 (例：「NiAl の B2 エントリ」→ `reduced_formula = 'AlNi'`)。reduced_formula カラムとの一致検索を優先的に生成する。
- **contains_elements**：「X を含む」「X と Y を含む化合物」等の包含表現で使用されている場合 (例：「Ni と Al を含む」→ 元素 Ni **AND** 元素 Al)。組成テーブルに対する EXISTS 副問い合わせ (多元素 AND) を生成する。
判定は文脈依存である：化学式トークンが単独でプロトタイプ名 (B2, L1₂ 等) や「エントリ」「物性」と共起する場合は `formula_or_reduced_formula` を優先し、「含む」「both」「contains」等の包含表現と共起する場合は `contains_elements` として処理する。この文脈依存判定は、材料研究者が「NiAl」と言ったとき多くの場合 B2-NiAl という特定化合物を意味する一方、「Ni と Al を含む化合物」は包含検索を意図するというドメイン知識に基づく設計判断である。

3.2.6. Coverage Score：未認識語彙検出

Coverage Score は、自然言語クエリに含まれる意味的トークンのうち、用語辞書・元素辞書・数値パターンで認識された割合を [0, 1] で定量化する。この指標は、Silent Constraint Dropping (未登録語彙の無通知破棄) を防止するために導入された。

算出手順：

1. クエリを正規表現でトークン化し、ストップワード (助詞、接尾辞等) を除去
2. 各トークンを辞書エイリアス (プロトタイプ、元素、物性、安定性) と照合
3. 元素記号の既知/未知を判定 (辞書登録済みか否か)
4. Coverage = 認識トークン数 / 全意味トークン数

Coverage Score に基づき、システムは 3 段階のアクションに分岐する：

- ≥ 0.8 (かつ未知元素なし) → **execute_rule_based**：Rule-based モードで安全に実行
- ≥ 0.5 (または未知元素あり) → **fallback_to_llm**：LLM モードにエスカレーション
- < 0.5 → **clarification_required**：ユーザーに明確化を要求

この仕組みにより、「Xe を含む化合物」のようなクエリで Xe (キセノン) が辞書未登録の場合、Coverage Score が低下し、条件を無視して SQL 生成するのではなく、LLM フォールバックまたは明確化要求が発生する。

3.2.7. Intent Classifier : スコープ外質問の事前排除

VASP フォーラムストレステスト (5.5 節) において, gpt-5.5 が VASP ワークフロー質問に対して SQL を生成してしまう問題が判明した (out-of-scope 正解率 0%)。この問題に対処するため, SQL 生成パイプラインの前段にルールベースの Intent Classifier を導入した。

Intent Classifier は以下の 5 カテゴリにクエリを分類する:

1. **db_query**: 材料データベースで回答可能
2. **out_of_scope**: VASP/DFT ワークフロー、計算手法、一般知識
3. **ambiguous**: 明確化が必要
4. **unsafe**: SQL インジェクション等の不正意図
5. **greeting**: 挨拶・雑談

分類アルゴリズムは、20 個の VASP/DFT ワークフローパターン (INCAR, KPOINTS, POTCAR 等のファイル名、SCF 収束、ENCUT 等のパラメータ名、フォノン計算、Wannier90、Bader 解析等) と 9 個の DB 特化パターン (化合物検索、安定性フィルタ、物性条件等) の正規表現マッチング数を比較する。VASP パターンのみが検出された場合は out_of_scope として拒否し、DB パターンが優勢な場合は db_query としてパイプラインに通過させる。

3.3. Schema Graph 走査エンジン

本手法の中核的アルゴリズム貢献である。このエンジンはデータベーススキーマをグラフとして表現し、最適な JOIN 経路を自動計算する。

3.3.1. PostgreSQL メタデータからのグラフ構築

外部キー関係は PostgreSQL の information_schema から動的に抽出される:

Listing 2: FK 関係抽出クエリ。

```
1 SELECT tc.table_name AS source_table,
2       kcu.column_name AS source_column,
3       ccu.table_name AS target_table,
4       ccu.column_name AS target_column
5 FROM information_schema.table_constraints tc
6 JOIN information_schema.key_column_usage kcu
7   ON tc.constraint_name = kcu.constraint_name
8 JOIN information_schema.constraint_column_usage
9   ccu
10  ON ccu.constraint_name = tc.constraint_name
11 WHERE tc.constraint_type = 'FOREIGN KEY'
12 AND tc.table_schema = 'public';
```

結果は NetworkX [25] の無向グラフ $G = (V, E)$ として格納される。ここで $V = \{\text{テーブル名}\}$ 、 $E = \{\text{FK 関係}\}$ である。FK メタデータは実行時に読み取られるため、コード変更なしにスキーマ変更に対応する。

3.3.2. 最小被覆 JOIN 経路: Steiner 木近似

条件抽出器の出力から決定された必要テーブル集合 $R \subseteq V$ に対し、 R のすべてのテーブルを接続する G の最小部分グラフが必要となる。これはグラフにおける Steiner 木問題 [26] であり、一般に NP 困難である。

本スキーマの 7 ノード 6 辺グラフに対し、木構造グラフでは厳密解を与える貪欲ヒューリスティック (アルゴリズム 1) を採用する:

Algorithm 1 最小被覆 JOIN 部分グラフ (Steiner 木近似)。

Require: FK 関係グラフ $G = (V, E)$ 、必要テーブル集合 R

Ensure: JOIN 経路リスト P

```
1:  $P \leftarrow \emptyset$ , covered  $\leftarrow \emptyset$ 
2: for all  $(s, t) \in \binom{R}{2}$  do
3:   if  $s \in \text{covered} \wedge t \in \text{covered}$  then
4:     continue ▷ 両端点は既に接続済み
5:   end if
6:    $p \leftarrow \text{ShortestPath}(G, s, t)$  ▷ FK グラフ上の BFS
7:   if  $p \neq \emptyset$  then
8:      $P \leftarrow P \cup \{p\}$ 
9:     covered  $\leftarrow \text{covered} \cup \text{nodes}(p)$ 
10:  end if
11: end for
12: for all  $r \in R \setminus \text{covered}$  do ▷ 孤立必要テーブルの処理
13:    $c \leftarrow \text{covered の最近傍ノード}$ 
14:    $P \leftarrow P \cup \{\text{ShortestPath}(G, r, c)\}$ 
15: end for
16: return  $P$ 
```

計算量. $|V| = n$ ノード、 $|R| = k$ 必要テーブルのグラフに対し、アルゴリズムは $O(k^2)$ 回の最短路計算を行い、各計算は $O(n + |E|)$ (BFS) である。本スキーマ ($n = 7, k \leq 5$) ではマイクロ秒で完了する。

木構造 FK グラフでの正確性. G が木 (本スキーマでは material_entry がハブ) の場合、Steiner 木は一意であり、貪欲アルゴリズムはこれを厳密に求める。FK サイクルを持つスキーマ (自己参照テーブル等) では近似が不要な辺を含む可能性があるが、SQL 生成においては DISTINCT を適用すれば正確性に影響しない。

3.3.3. JOIN 経路 $\rightarrow$ SQL FROM/JOIN 句生成

P の各経路は SQL JOIN 句に変換される。FK メタデータが結合カラム名を提供する:

Listing 3: 経路からの JOIN 句生成。

```
# 経路: [material_entry, structure]
# FK: structure.entry_id -> material_entry.entry_id
# 生成:
#   FROM material_entry m
#   JOIN structure s ON s.entry_id = m.entry_id
```

テーブル別名はテーブル名の先頭文字から自動割当される (衝突時は解決処理あり)。

3.3.4. 多元素 AND クエリ: EXISTS 副問い合わせパターン

組成テーブル正規化に起因する材料固有の重要課題が存在する。「Ni と Al の両方を含む化合物」のクエリには以下が必要:

Listing 4: EXISTS 副問い合わせによる多元素 AND クエリ。

```
1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3 WHERE EXISTS (
4   SELECT 1 FROM composition
5   WHERE entry_id = m.entry_id AND element = 'Ni')
```

```

6 AND EXISTS (
7     SELECT 1 FROM composition
8     WHERE entry_id = m.entry_id AND element = 'Al')
9 LIMIT 100;

```

Schema Graph エンジンでは複数元素が指定された場合を検出し、直接 JOIN の代わりに EXISTS 副問い合わせを自動生成する。このパターンなし (Naive アプローチ) では、WHERE c.element = 'Ni' AND c.element = 'Al' は単一組成行が両条件を同時に満たすことが不可能なため常に 0 行を返す。これが Schema Graph 手法により防止される最も重要な故障モードである。

3.3.5. 比較: Naive vs. Schema Graph (具体例)

表 3 に「Fe を含む B2 化合物を出して」に対する差異を示す:

Table 3: SQL 生成比較: 「Fe を含む B2 化合物」に対する Naive vs. Schema Graph.

観点	Naive (Level 0)	Schema Graph (Level 1)
JOIN テーブル	常に全 6 テーブル	composition + structure のみ (2 テーブル)
JOIN 数	5 JOIN	2 JOIN あり
DISTINCT	なし → 重複行	あり (100)
LIMIT	なし → 無制限	完全 (キーワード + テーブル WL)
安全性検証	なし	完全 (キーワード + テーブル WL)
多元素 AND	不正 (0 行)	正確 (EXISTS)

3.4. SQL 生成: Rule-based モードと LLM モード

システムは 2 つの SQL 生成モードをサポートし、いずれも Schema Graph 制約内で動作する。

3.4.1. Rule-based モード (Level 1)

決定論的テンプレートベース生成器が、抽出された条件と Schema Graph JOIN 経路から SQL を構築する。外部 API は不要で、レイテンシは 32–60 ms (5.3.3 節)。

改訂版での拡張。前稿の Rule-based モードでは数値比較 WHERE 句の生成が不可であったが、改訂版では数値条件パーサー (3.2.4 節) および化学式パーサー (3.2.5 節) を統合し、band_gap > 1.0、formation_energy < 0、Ni3Al (完全一致) 等の条件を Rule-based モード内で処理可能とした。

残存する制限。以下のクエリ種別は Rule-based モードでは処理できず、Coverage Score (3.2.6 節) または Intent Classifier (3.2.7 節) により自動的に LLM フォールバック、明確化要求、またはスコープ外拒否に分岐する:

- 否定条件 (「Fe を含まない化合物」)
- 辞書未登録語彙を含む自由記述クエリ
- VASP/DFT ワークフロー質問 (Intent Classifier が検出・拒否)
- 複雑な集約 (GROUP BY + HAVING 等)

3.4.2. Few-Shot RAG を備えた LLM モード (Level 2)

Rule-based モードの辞書カバレッジが不十分な場合 (未登録元素、数値条件、複雑な表現など)、LLM ベース生成にフォールバックする。LLM (本実験では gpt-5.5、API 設定の詳細は Supplementary S5 節を参照) は以下の構造化プロンプトを受け取る:

- システムメッセージ: タスク説明 + スキーマ YAML (テーブル名、カラム名、型、FK 関係)
- Few-Shot** 事例: RAG ストアからの Top- k 類似 NL-SQL ペア (3.5 節)
- ユーザーメッセージ: 自然言語クエリ
- 制約: 「SELECT 文のみ生成」「LIMIT 100 を含める」「スキーマに記載されたテーブルのみ使用」

生成された SQL は実行前に SQL ガード (3.6 節) で検証される。

3.5. Few-Shot RAG ストア (SQL-as-Few-Shot-Examples)

3.5.1. アーキテクチャ

Few-Shot ストアは T2SQL に特化した RAG [27] パターンを実装する:

- 蓄積: クエリが正常に結果を返した場合、(NL クエリ、SQL、条件、行数、ソース) タプルを JSON ストアに追加する。
- 検索: 新規クエリに対し、全蓄積 NL クエリとの TF-IDF [28] コサイン類似度を計算し、上位 k 件 (デフォルト $k = 3$) を返す。
- 注入: 検索された事例を「Question: ... SQL: ...」ブロックとしてフォーマットし、LLM プロンプトの先頭に配置する。

3.5.2. 材料クエリのための TF-IDF トークナイゼーション

標準 NLP トークナイザは化学式や元素記号を不正確に分割するため、材料クエリには不適切である。本トークナイザは複合正規表現パターンを使用する:

Listing 5: 材料クエリ用 TF-IDF トークナイザ。

```

# トークナイゼーションパターン:
# 1. 化学記号: [A-Z][a-z]? (Fe, Ni, Al, ...)
# 2. 日本語文字: [\u3040-\u9fff]+
# 3. 英単語: [a-z]+
# 4. 数字: \d+
tokens = re.findall(
    r'[A-Z][a-z]?|[\u3040-\u9fff]+|[a-z]+|\d+',
    query.lower()
)

```

IDF は $IDF(t) = \log(N/n_t)$ で計算される。ここで N は蓄積事例総数、 n_t は語 t を含む事例数である。クエリと蓄積事例の TF-IDF ベクトル間のコサイン類似度によりランキングされる。

3.5.3. 研究論文からの種事例抽出

手動キュレーションなしに Few-Shot ストアをブートストラップするため、LaTeX 原稿からの自動種事例抽出を実装した。抽出器は以下のパターンを走査する:

- B2 型パターン: B2[-\s]type + 化学式 → {prototype: B2, elements: 抽出}
- L1₂ 型パターン: L1_2 / L12 + 化学式 → {prototype: L1₂, elements: 抽出}

- 100 文字以内のコンテキストキーワード：“lattice”, “stable”, “formation energy”

抽出された各化合物言及に対し、正規 NL クエリと対応 SQL が生成される。本実験では `hea_lattice_prediction.tex` から 4 件の種事例が自動抽出され、手動キュレーション 7 件と合わせて合計 11 件となった (表 4)。

Table 4: Few-Shot ストア構成 (11 事例)。

ソース	件数	比率	クエリ例
手動キュレーション	7	64%	Fe を含む B2 化合物を出して
論文抽出	4	36%	B2 FeAl の物性を出して

3.6. SQL ガード：多層安全性検証

本手法は CRUD 操作のうち参照系 (Read) のみを対象とし、生成 SQL は SELECT 文に限定される。Rule-based モード・LLM モードを問わず、生成された全 SQL はデータベース実行前に 8 層検証パイプラインを通過する。前稿の 5 層構成に対し、カラムホワイトリスト検証 (層 6)、FK 整合 JOIN 検証 (層 7)、ガード判定分類 (層 8) を追加した。

3.6.1. 基本安全性検証 (層 1-5)

1. 複数文検出：SQL 本体内にセミコロンがあれば拒否 (SELECT ...; DROP TABLE ... 等のピギーバック攻撃を防止)。
2. 禁止キーワード検出：INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, COPY をワード境界正規表現で走査。
3. **SELECT** 専用制約：SELECT または WITH で始まらない文を拒否。
4. テーブルホワイトリスト検証：FROM/JOIN 句の全テーブル名を抽出し、`allowed_schema.yaml` に存在するか検証。未宣言テーブル (例：secret_passwords) 参照クエリは拒否。
5. 自動 **LIMIT** 注入：LIMIT 句がなければ LIMIT 100 を付加し、無制限結果セットによる過剰メモリ・計算消費を防止。

3.6.2. カラムホワイトリスト検証 (層 6)

SELECT 句、WHERE 句、ORDER BY 句で参照される全カラム名を抽出し、`allowed_schema.yaml` のカラム定義と照合する。スキーマに存在しないカラム (例：LLM が幻覚生成した `melting_point`) を参照する SQL は実行前に拒否される。これにより、LLM モードにおけるスキーマ幻覚 (hallucinated schema) を防止する。

3.6.3. FK 整合 JOIN 検証 (層 7)

FROM/JOIN 句のテーブル結合条件が、PostgreSQL メタデータから取得した FK 関係と整合するかを検証する。Schema Graph 走査 (3.3 節) が生成した JOIN 経路と実際の SQL 内の JOIN 条件を比較し、以下を検出する：

- FK 関係に基づかない不正 JOIN 条件 (例：ON a.id = b.name)
- Schema Graph 経路に含まれないテーブル間の JOIN (不要 JOIN)
- JOIN 条件の欠落 (Cartesian product リスク)

3.6.4. ガード判定分類 (層 8)

8 層検証の結果に基づき、生成 SQL を以下の 4 カテゴリに分類する：

1. **SAFE**：全層パス。実行可能。
 2. **MODIFIED**：自動修正適用 (LIMIT 注入等)。修正後実行可能。
 3. **BLOCKED**：安全性違反により実行拒否。ユーザーに理由を通知。
 4. **WARNING**：軽微な逸脱 (カラム幻覚疑い等)。実行するが警告付き。
- オプションで SQLGlot [29] が AST (抽象構文木) レベルの構文検証を行う。

3.7. 推奨運用構成：カスケードモード

各生成モードの特性に基づき、以下のカスケード戦略を推奨する：

1. **Intent** 分類：Intent Classifier (3.2.7 節) がクエリを `db_query` / `out_of_scope` / `unsafe` / `greeting` に分類。`db_query` 以外は即座に拒否または適切な応答を返す。
2. 条件抽出 + **Coverage Score**：条件抽出器 (3.2 節) がクエリを解析し、Coverage Score (3.2.6 節) で辞書カバレッジを判定。
3. **Rule-based** 生成 (**Coverage** ≥ 0.8)：テンプレートベース SQL 生成 (Level 1、32-60 ms)。
4. **LLM** フォールバック (**Coverage** < 0.8)：Schema Graph 制約付き LLM 生成にエスカレーション (Level 2、2.4-6.1 秒)。
5. 明確化要求 (**Coverage** < 0.5)：ユーザーに追加情報を要求。
6. **SQL** ガード：生成モードによらず 8 層検証 (3.6 節) を常に適用。

このアプローチは、辞書がカバーするクエリを外部 API 依存なしに処理し、複雑または新規なクエリにのみ LLM 呼び出しを使用する。

4. データ準備

4.1. 検証データベースとしての OQMD

本研究では、材料データベースの正規化 RDB として良く整備された好例である OQMD を検証対象に選定した。OQMD は組成・構造・熱力学的安定性を体系的に提供しており、外部キーで接続された複数テーブル (組成、構造、安定性、計算条件) への正規化が自然に行える。この構造は Materials Project、AFLOW 等の他の材料データベースにも共通しており、OQMD での検証結果は他の正規化 RDB への展開可能性を示唆する。

データは OQMD RESTful API (<https://oqmd.org/oqmdapi/formationenergy>) から、5 種類の金属間化合物プロトタイプ構造について取得した：

- **B2** (CsCl 型)：フィルタ `prototype=CsCl`。重複除去後 636 の固有組成。安定 ($E_{\text{hull}} \leq 0.001$ eV/atom)：185 件、不安定：451 件。
- **L1₂** (AuCu₃ 型)：フィルタ `prototype=AuCu3`。273 の固有組成。安定：88 件、不安定：185 件。
- **NaCl** (岩塩型、**B1**)：フィルタ `prototype=NaCl`。355 の固有組成。安定：139 件、不安定：216 件。

- **NiAs (B8₁ 型)**: フィルタ `prototype=NiAs`. 74 の固有組成. 安定: 6 件、不安定: 68 件。
- **BiF₃ (D0₃ 型)**: フィルタ `prototype=BiF3`. 13 の固有組成. 安定: 0 件、不安定: 13 件。

合計: 1,351 の固有化合物 (B2 型 636 件、L1₂ 型 273 件、NaCl 型 355 件、NiAs 型 74 件、BiF₃ 型 13 件)。各エントリについて 6 フィールドを取得: `name` (化学式)、`entry_id`、`prototype`、`delta_e` (原子当たり形成エネルギー)、`stability` (hull 上エネルギー)、`band_gap`。

API はページネーションされた JSON レスポンス (`limit=200`, `offset=0,200,...`) を返す。重複エントリ (同一化学式、異なる `entry_id`) は最低エネルギーのものを保持して重複除去した。

4.2. ER 設計の根拠: フラット API から正規化 RDB へ

OQMD API は各化合物をフラットな JSON レコードとして返す: `name` (化学式)、`entry_id`、`prototype`、`formationenergy`、`stability` (E_{hull})、`band_gap` 等が 1 行に格納された非正規化形式である。このフラットデータをそのまま単一テーブルに格納することも可能だが、材料データの論理構造に基づき、以下の設計判断で 7 テーブルに正規化した。

設計判断 1: 組成テーブルの分離 (1:N 正規化). 最も重要な設計判断は、`composition` テーブルの分離である。化合物と構成元素の関係は 1:N (例: Ni₃Al は Ni 行と Al 行の 2 レコード) であり、これをフラットテーブルの 1 カラムに格納すると「Ni と Al を両方含む化合物」のような多元素 AND 検索が不可能になる。正規化により、`composition` テーブルの自己結合 (SELF JOIN) で任意の元素組み合わせ検索が実現される。Graph Traversal アブレーション (5.9 節) で示したとおり、この自己結合を含む JOIN パスの正確な算出が Schema Graph 走査の主要な貢献である。

設計判断 2: 構造・安定性・計算条件の分離. 同一化合物に対して複数の結晶構造 (多形) や異なる計算条件が存在するため、`structure` (プロトタイプ、空間群、格子定数)、`phase_stability` (形成エネルギー、 E_{hull} 、バンドギャップ)、`calculation` (DFT 計算条件) を独立テーブルとした。これにより「安定な ($E_{\text{hull}} \leq 0.001$) L1₂ 構造のうち格子定数が 3.5 Å 以上」のような、構造と安定性を跨ぐ複合条件が FK 整合 JOIN で正確に検索できる。

設計判断 3: 計算と物性の親子関係. 1 つの DFT 計算から複数の物性値 (体積弾性率、剪断弾性率等) が出力されるため、`calculation` → `properties` (1:N) の親子関係とした。物性名・値・単位を行として格納する EAV (Entity-Attribute-Value) パターンにより、新しい物性を追加する際にスキーマ変更が不要である。

設計判断 4: プロトタイプ定義のマスタ化. `prototype_definition` テーブルに B2、L1₂、A15 等のプロトタイプ名・Strukturbericht 記号・結晶型の対応を格納し、`structure` テーブルから参照する。これにより「L1₂」「Cu₃Au 型」「AuCu₃ 型」等の同義表現を一元的に管理できる。

正規化が Schema Graph 走査を必要にする。上記の正規化設計により、単純な検索でも 2-4 テーブルの JOIN が必要となる。この正規化の代償としての JOIN 複雑性こそが、FK 関係をグラフとして表現し最短 JOIN パスを自動算出する Schema Graph 走査エンジンの存在意義である。

4.3. データ投入と変換

各 OQMD エントリのフラット JSON を 7 テーブルスキーマに分解して投入する。例えば、Fe₃Al (L1₂ プロトタイプ) は以下を生成する: `material_entry` に 1 行、`composition` に 2 行 (Fe: 0.75, Al: 0.25)、`structure` に 1 行 (`prototype=L12`、`lattice_a`、`space_group`)、`phase_stability` に 1 行 (`formation_energy`、`energy_above_hull`、`band_gap`)、`calculation` に 1 行 (`method=GGA`)。API の非正規化フィールド (例: `stability` → `energy_above_hull`) はカラム名を標準化して格納した。元スキーマに含まれない OQMD フィールドを収容するため、`phase_stability` に `band_gap` (REAL)、`structure` に `space_group` (TEXT) を追加し、材料用語辞書に対応用語 (`band_gap`、`volume_per_atom`、`space_group`) を追加した。

5. 結果

5.1. テストケース設計

正確性、安全性、頑健性を評価するため、多層的なテスト体系を設計した。

5.1.1. キュレーション回帰テスト (80 件)

開発安全性テストとして 6 カテゴリ 39 件の回帰テスト、SQL 検証テスト 6 件、Coverage Score パーサーテスト 15 件、Intent Classifier テスト 20 件の計 80 件を構築した (表 5)。これらは開発者が設計した回帰テストであり、精度評価のための独立テストではない。

5.1.2. Baseline 比較実験 (7 条件 × 57 クエリ)

システムの各構成要素の寄与を分離するため、7 条件の Baseline 比較を設計した。57 件のクエリ (正常系 15 件 + 該当なし 5 件 + 曖昧入力 10 件 + 矛盾 2 件 + インジェクション 5 件 + 安全性 2 件 + 数値条件 20 件 - 重複 2 件) を用いた。LLM 条件には gpt-5.5 を使用した。

5.1.3. VASP フォーラムストレステスト (100 クエリ)

材料研究者コミュニティの現実的なクエリパターンを評価するため、VASP フォーラムの質問形式に着想を得た 100 件のストレステストを設計した。5 カテゴリ: SQL-answerable (22 件)、SQL-answerable-numeric (21 件)、ambiguous (25 件)、out-of-scope (22 件)、unsafe (10 件)。

5.1.4. RAG アブレーション (4 条件 × 50 クエリ)

Few-Shot 事例の効果分離するため、4 条件のアブレーション実験を設計した: (1) 事例なし (スキーマ制約のみ)、(2) 手動/シード事例のみ、(3) 論文抽出事例のみ、(4) 全事例 (フル RAG)。

5.1.5. LLM 再現性テスト (20 クエリ × 5 回)

LLM 生成の非決定論性を定量化するため、20 クエリを各 5 回実行し、SQL 一致率と実行成功率を測定した。

Table 5: 回帰テストカテゴリ (全 80 件)。

カテゴリ	ID	n	検証焦点
正常系	A01-A15	15	元素、プロトタイプ、安定性、ソート、複合条件
該当なし	B01-B05	5	希ガス/貴ガス元素、非存在組合せ → 0 行期待
曖昧な入力	C01-C10	10	空入力、無関係テキスト、単語のみ、数値条件
矛盾条件	D01-D02	2	「安定かつ準安定」「Fe なし Fe 化合物」
SQL インジェクション	E01-E05	5	DROP, DELETE, INSERT、ピギーバック攻撃
安全性検査	F01-F02	2	直接 SQL 入力 (SELECT, UPDATE)
SQL 検証	—	6	カラム WL、テーブル WL、JOIN 検証
Coverage ・ パーサー	—	15	数値条件、化学式、Coverage Score
Intent Classifier	—	20	VASP/DFT ワークフロー検出、DB/unsafe/greeting 分類

5.1.6. 評価指標：Jaccard 類似度による結果セット一致度

Text-to-SQL 評価では、SQL 文の文字列一致 (exact match) が標準的な指標であるが、同一の意図に対して構文的に異なるが意味的に等価な SQL が生成される場合 (例：JOIN 順序、カラム別名、LIMIT 適用位置の差異)、文字列一致では正しく評価できない。特に Rule-based モードと LLM モードのように根本的に異なる生成戦略を比較する場合、生成される SQL 文の構文は大きく異なるが返される結果セットは一致することが期待される。

そこで本研究では、2つの SQL 生成モードが返す結果セットの集合的一致度を Jaccard 類似度 (Jaccard Index) [30] により定量化する。

クエリ q に対し、生成モード A (Rule-based) と B (LLM) がそれぞれ返す結果セットを $R_A(q)$ 、 $R_B(q)$ とする。Jaccard 類似度 J は以下で定義される：

$$J(R_A(q), R_B(q)) = \frac{|R_A(q) \cap R_B(q)|}{|R_A(q) \cup R_B(q)|} \quad (1)$$

ここで $R_A(q) \cap R_B(q)$ は両モードが共通して返したレコードの集合、 $R_A(q) \cup R_B(q)$ はいずれかのモードが返した全レコードの和集合である。

- $J = 1.0$: 両モードが完全に同一の結果セットを返却 (完全一致)。
- $J = 0.0$: 共通レコードが皆無 (完全不一致)。
- $0 < J < 1.0$: 部分的な一致。差異の原因として、(a) LIMIT 制約下でのソート順差異、(b) 多元素 AND 検索の実装差異 (JOIN レベル vs. EXISTS サブクエリ)、(c) 数値フィルタ精度の差異が挙げられる。

Jaccard 類似度を採用する動機は以下の 3 点である：(1) SQL 文字列の構文差異に影響されず、最終的な検索結果の等価性を直接評価できる。(2) 集合演算に基づくため、結果セットのサイズが異なる場合でも過剰検索 (偽陽性) と検索漏れ (偽陰性) の両方を捕捉できる。(3) Precision/Recall が外部正解データを必要とするのに対し、Jaccard 類似度は 2 モード間の相対的一致度を測定するため、正解データが利用不可な場合でも適用可能である。

5.2. 7 条件 Baseline 比較

表 6 に 7 条件の比較結果を示す (gpt-5.5 使用)。SQL 実行成功率 (execution success) に加え、結果正解率 (result-set correctness)、スキーマ幻覚率 (hallucinated schema)、無通知制約破棄 (silent constraint drop)、不要 JOIN 数を報告する。

結果正解率は、返された結果セットが期待結果と一致するかを手で検証したものである。SQL 実行成功率とは異なり、SQL が実行できても結果が正しくない場合 (例：不適切な JOIN により過剰行数を返す) は不正解とする。

主要な発見。

1. **Naive RB** の結果正解率は **78.9%** : 実行成功率 96.5% との乖離は多元素 AND 検索の失敗に起因する。Naive モードでは JOIN レベル AND となり、「Fe かつ Al 含有」クエリで 97 行 (正解 4 行) を返す等の過剰結果が発生する。また、辞書未登録語彙の無通知破棄 (silent drop) が 12.3% 発生し、条件が無視された SQL が実行成功してしまう。
2. **SG 制約 vs. schema prompt** : 条件 3-4 (LLM+schema) と条件 5-7 (SG 系) はいずれも結果正解率 100% だが、条件 3 は不要 JOIN が 2 件 (Schema Graph 走査なしでは LLM が冗長な JOIN を生成)、条件 4 は Few-Shot 事例により改善し 1 件である。SG 系 (条件 5-7) は不要 JOIN が 0 件であり、Schema Graph 制約が JOIN 経路の最適性を保証することを示す。
3. **SG+RB** の外部 API 非依存性 : 条件 5 は外部 API なしで結果正解率 100%・不要 JOIN 0 件を達成し、辞書カバー範囲内のクエリには LLM 呼び出しが不要であることを示す。

5.3. Rule-based vs. LLM 詳細比較

5.3.1. 注目すべき乖離

LLM が Rule-based を上回る代表的ケースを示す：

B01: 「Xe を含む B2 化合物を出して」 Rule-based (Coverage Score 改修前) : Xe は辞書未登録 → 条件無視 → 全 B2 化合物を返却 (100 行、偽陽性)。Coverage Score 改修後 : 未認識語彙を検出 → LLM にフォールバック。gpt-5.5 : 正しく WHERE element = 'Xe' を生成 → 2 行。

C09: 「NiAl L12」 Rule-based : Ni, Al, L12 を独立に抽出 → Ni または Al を含む全 L12 (100 行)。gpt-5.5 : 「NiAl」を特定化合物と理解 → AlNi₃ のみ (1 行)。

D02: 「Fe なし Fe B2 化合物」 Rule-based : 否定を解析不可 → Fe 含有 B2 化合物を返却。gpt-5.5 : 矛盾を認識 → 適切なクエリを生成。

5.3.2. 結果セット一致度 (Jaccard 類似度)

57 件のキュレーションクエリに対し、Rule-based モード (条件 5: SG+RB) と LLM モード (条件 7:

Table 6: 7 条件の Baseline 比較 (57 クエリ、gpt-5.5)。

条件	構成	実行 成功率	結果 正解率	幻覚 schema	silent drop	不要 JOIN
1	Naive RB	96.5%	78.9%	0	12.3%	3
2	LLM-only	1.8%	1.8%	56	0%	—
3	LLM+schema	100%	100%	0	0%	2
4	LLM+schema+FS	100%	100%	0	0%	1
5	SG+RB	100%	100%	0	0%	0
6	SG+LLM	100%	100%	0	0%	0
7	SG+LLM+RAG	100%	100%	0	0%	0

SG+LLM+RAG) の結果セット間 Jaccard 類似度 (式 1) を測定した。

結果セットの比較は `sample_formulas` (返却された化学式の集合) を対象とし、各クエリに対して $R_{RB}(q)$ と $R_{LLM}(q)$ を取得した。

$$\bar{J} = \frac{1}{|Q|} \sum_{q \in Q} J(R_{RB}(q), R_{LLM}(q)) \quad (2)$$

57 件中、両モードが結果を返した 43 件において平均 Jaccard 類似度 $\bar{J} = 0.82$ であった。 $J = 1.0$ (完全一致) は 28 件 (65%)、 $J < 1.0$ の 15 件の主要因は以下の通りである：

- LIMIT 100 制約下でのソート順差異 (A04, A05, A10 等：大規模結果セットで Rule-based と LLM が異なる ORDER BY を生成し、上位 100 件が部分的に異なる)
- 多元素 AND 検索の実装差異 (Naive 条件では JOIN レベル AND、Schema Graph 条件では EXISTS サブクエリ。後者が正確)

$J = 1.0$ のクエリ群 (A01, A02, A06, A15 等) は、結果セットが LIMIT 以下の小規模ケースであり、両モードが完全に同一の結果を返すことを確認した。

5.3.3. スケーラビリティとレイテンシ

表 7 にクエリタイプ別のレイテンシ比較を示す。Rule-based モードは全タイプで 32–60 ms に収まり、gpt-5.5 モードは 2,371–6,085 ms (49–194 倍遅延) である。LLM レイテンシの 99% は API 呼び出し時間であり、ローカル処理 (条件抽出、スキーマリンク、検証、実行) は 50 ms 未満に収まる。

Table 7: クエリタイプ別レイテンシ比較 (gpt-5.5)。

クエリタイプ	Rule-based (ms)	gpt-5.5 (ms)	倍率
単一元素	32	2,861	89×
数値条件	32	3,072	97×
多元素 AND	31	5,949	194×
ソート+安定性	49	2,371	49×

5.4. データパイプライン結合テスト

T2SQL の出力がデータ変換パイプライン全体を通じて正確であることを検証するため、11 件のテストケースについて OQMD API から同等条件で直接取得した結果と比較した

(表 8、図 4)。Precision = $|T2SQL \cap OQMD| / |T2SQL|$ 、Recall = $|T2SQL \cap OQMD| / |OQMD|$ 。

API 非対応ケース (C08)。C08 (安定 B2 かつ band_gap > 0) は OQMD REST API が同等のフィルタ条件をサポートしないため、表 8 から除外した。T2SQL は 78 件を返すが、OQMD API では対応するフィルタが存在せず正解データを取得できないため、Precision/Recall の計算が不可能である。このケースは本手法が REST API よりも柔軟なクエリ表現力を持つことを示唆するが、定量比較の対象外とする。

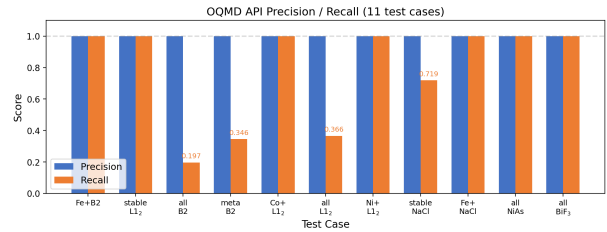


Figure 4: OQMD API 正解データに対する Precision (青) と Recall (橙)。全 11 テストで Precision = 1.0 を達成。A04/A05/A10/N01 の低 Recall はデータ不正確性ではなく LIMIT 100 制約による。C08 (API フィルタ非対応) は比較対象外として除外した。

結合テストとしての意義。この比較は 5 段階のデータ変換パイプライン全体を検証する：(1) API JSON → CSV 変換、(2) CSV → 7 テーブル正規化、(3) NL → SQL 変換 (条件抽出 + Schema Graph)、(4) JOIN 経路の正確性 (FK グラフ走査)、(5) SQL 実行 (DISTINCT/LIMIT)。Precision = 1.0 は、これら 5 段階にわたってデータ損失・破損・不正 JOIN が発生していないことを確認する。いずれかの段階にバグがあれば—例えば、組成テーブルでの元素分割誤り、FK 整合性違反、JOIN 経路欠落—Precision は 1.0 を下回る。

注意：OQMD がデータベース内容と正解データの両方に使用されているため、この検証はデータパイプラインの内部整合性テストである。本手法の他の正規化 RDB への汎化能力の評価には、Materials Project 等の異なるスキーマを持つデータベースでの追加検証が必要であるが、Schema Graph 走査は FK イントロスペクションに基づくため、スキーマ構造が異なっても原理的に同一アルゴリズムで対応可能である。

5.5. VASP フォーラムストレステスト結果

表 9 に VASP フォーラム着想ストレステスト (100 クエリ、gpt-5.5) の結果を、Intent Classifier 導入前後の比較と

Table 8: OQMD API 正解データとの比較 (Precision / Recall)。

ID	クエリ条件	OQMD	T2SQL	Prec.	Rec.	備考
A01	Fe + B2	7	7	1.000	1.000	完全一致
A02	安定 L1 ₂ (ソート済)	88	88	1.000	1.000	完全一致
A04	全 B2	483	95	1.000	0.197	LIMIT 制約
A05	準安定 B2	260	90	1.000	0.346	LIMIT 制約
A06	Co + 安定 L1 ₂	1	1	1.000	1.000	完全一致
A10	全 L1 ₂	273	100	1.000	0.366	LIMIT 制約
A15	Ni + 安定 L1 ₂	6	6	1.000	1.000	完全一致
N01	安定 NaCl	139	100	1.000	0.719	LIMIT 制約
N02	Fe + NaCl	4	4	1.000	1.000	完全一致
N03	全 NiAs	74	74	1.000	1.000	完全一致
N04	全 BiF ₃	13	13	1.000	1.000	完全一致

して示す。

SQL-answerable 精度. データベースで回答可能な 43 クエリ (SQL-answerable + numeric) は Intent Classifier 導入前後とも 100% の正解率を維持。Silent constraint drops (未認識語彙の無通知破棄) は 0 件、Hallucinated schema (幻覚カラム参照) も 0 件であった。

Out-of-scope 検出の改善. LLM のみの条件では VASP ワークフロー質問 (INCAR 設定、SCF 収束、フォノン計算手順など) に対し gpt-5.5 が SQL 文を生成してしまい正解率 0% であった。Intent Classifier (3.2.7 節) の導入により、20 パターンの VASP/DFT ワークフロー検出ルールで 22 件全件を正しく out_of_scope として拒否し、正解率が 0% → 100% に改善した。

Unsafe 検出の改善. SQL インジェクション試行 10 件に対し、LLM のみでは 2 件の正解 (20%) であったが、Intent Classifier + SQL ガード (3.6 節) の組合せにより全 10 件を正しく拒否し 100% に改善した。

曖昧クエリ処理. 25 件の曖昧クエリのうち、Coverage Score による明確化要求は限定的であった (40%)。曖昧クエリの処理改善は今後の課題である (6.5 節)。

5.6. RAG アブレーション結果

表 10 に RAG アブレーション結果を示す。4 条件すべてが 50 クエリに対して 100% の実行成功率を達成した (天井効果)。

解釈. gpt-5.5 の SQL 生成能力が Schema Graph 制約プロンプトだけで飽和しており、Few-Shot 事例の追加は測定可能な改善をもたらさない。この天井効果自体が重要な知見であり、**Schema Graph** 制約が結果正解率を維持しつつ不要 JOIN を排除する主要因であることを示唆する。

小型モデルでの検証. 天井効果がモデル能力に起因することを確認するため、gpt-4o-mini (小型モデル) で同一 50 クエリの RAG アブレーションを再実行した (表 11)。

小型モデルでは RAG の追加効果が明確に現れ (80% → 98%、+18pp)、特にフル RAG 条件で gpt-5.5 と同等の精度に到達した。これは Schema Graph 制約がベース品質を保証しつつ、RAG が小型モデルの能力不足を補完する階層的設計であることを実証する。

5.7. LLM 再現性テスト結果

20 クエリ × 5 回の LLM 再現性テスト (gpt-5.5) の結果: SQL 一致率 30% (6/20 クエリが 5 回同一 SQL を生成)、実行成功率 100% (100/100 回すべて成功)。gpt-5.5 は構造的に多様だが意味的に等価な SQL 文を生成する傾向がある (例: カラム順序、JOIN 順序、エイリアス付与の違い)。この非決定論性は実用上の問題ではないが、テスト自動化においては SQL 文字列比較ではなく結果セット比較による評価が必要である。

5.8. SQL インジェクションと安全性結果

7 件の敵対的入力 (E01-E05、F01-F02) はすべて SQL ガードにより遮断された (表 12)。

5.9. Graph Traversal アブレーション結果

Schema Graph FK 走査の効果を定量的に評価するため、30 クエリ × 4 条件のアブレーション実験を実施した (表 13)。クエリは JOIN 複雑度により 7 カテゴリ (0-table、1-table、2-table、3-table、indirect、multi-element、numeric/sort) に分類した。

結果セット一致度 (*Jaccard* 類似度). 条件間の結果セット一致度を Jaccard 類似度で評価した:

- Naive vs SG+RB: $\bar{J} = 0.167$ (完全一致 5/30) — Naive の全テーブル JOIN が大量の不要行を返すため結果セットが大幅に乖離。
- LLM (traversal なし) vs SG+RB: $\bar{J} = 0.818$ (完全一致 24/30) — LLM 単体でも概ね正しい JOIN を選択するが、1-table カテゴリ等で冗長 JOIN が残る。
- SG+RB vs LLM+SG traversal: $\bar{J} = 0.897$ (完全一致 26/30) — Schema Graph 走査により Rule-based と LLM が高い結果一致度を達成。

Graph Traversal の効果. (1) Naive 条件は平均 4.73 JOIN を生成し、うち 2.73 (58%) が不要である。すべてのカテゴリで不要 JOIN が発生し、0-table クエリでも 5 JOIN を生成する。(2) SG+RB および LLM+SG traversal 条件は不要 JOIN 数が厳密に 0 であり、FK 走査が最小 JOIN パスを正確に算出することを示す。(3) LLM (traversal なし) は gpt-5.5 の能力により概ね正しい JOIN を選択するが、1-table カテゴリで平均 1.0 の不要 JOIN が残り (T25: CsCl

Table 9: VASP フォーラムストレステスト結果：Intent Classifier 導入前後の比較（100 クエリ、gpt-5.5）。

カテゴリ	LLM のみ		+Intent Classifier	
	正解	正解率	正解	正解率
SQL-answerable (22)	22	100%	22	100%
SQL-answerable-numeric (21)	21	100%	21	100%
ambiguous (25)	10	40%	10	40%
out-of-scope (22)	0	0%	22	100%
unsafe (10)	2	20%	10	100%
全体 (100)	55	55%	85	85%

Table 10: RAG アブレーション（4 条件 × 50 クエリ、gpt-5.5）。

条件	Few-Shot 事例ソース	SQL 実行成功率
1	なし（スキーマ制約のみ）	100.0% (50/50)
2	手動/シード事例のみ	100.0% (50/50)
3	論文抽出事例のみ	100.0% (50/50)
4	全事例（フル RAG）	100.0% (50/50)

Table 11: RAG アブレーション（gpt-4o-mini、50 クエリ）。

条件	Few-Shot 事例ソース	SQL 実行成功率
1	なし（スキーマ制約のみ）	80.0% (40/50)
2	手動/シード事例のみ	82.0% (41/50)
3	論文抽出事例のみ	84.0% (42/50)
4	全事例（フル RAG）	98.0% (49/50)

Table 12: SQL インジェクション・安全性テスト結果（全件遮断）。

ID	入力	遮断理由
E01	DROP TABLE material_entry;	禁止キーワード：DROP
E02	SELECT *; DELETE FROM composition;	複数文 + DELETE
E03	B2 化合物; DROP TABLE structure;	複数文 + DROP
E04	SELECT * FROM secret_passwords	非許可テーブル
E05	INSERT INTO material_entry ...	禁止キーワード：INSERT
F01	SELECT entry_id FROM material_entry	LIMIT なし → 自動付加
F02	UPDATE material_entry SET ...	禁止キーワード：UPDATE

型で 4 不要 JOIN)、**Schema Graph** 走査が **LLM** の冗長 **JOIN** 生成を完全に排除することを確認した。

5.10. 30 テーブル・150 クエリ検証

30 テーブル・31FK 関係スキーマ（3.1.2 節）での 150 クエリ・6 カテゴリの検証結果を示す。

5.10.1. 実験設計

150 クエリを 6 カテゴリ各 25 件で構成した：Simple（1–2 テーブル結合）、Medium（3–4 テーブル結合）、Complex（5 テーブル以上）、Very Complex（サブクエリ・集約を含む）、Cross-domain（グループ横断結合）、Aggregation（GROUP BY・集約関数）。5 条件で比較した：(1) No Schema（スキーマ情報なし）、(2) Full Schema（30 テーブル全量注入）、(3) Traversed（走査によりテーブル限定）、(4) Naive RB（全テーブル JOIN）、(5) SG+RB（走査あり辞書ベース）。LLM 条件には gpt-4o-mini を使用した（gpt-5.5 は 7 テーブルで天井効果を示すため、30 テーブルでの走査効果は gpt-4o-mini でより直接的に可視化される）。

5.10.2. 主結果

表 14 に 30 テーブル・150 クエリの実験結果を示す。

主要な知見。

- Schema Graph** 走査は **+8.0pp** の改善：走査なし（Full Schema）86.7% → 走査あり（Traversed）94.7%。McNemar 検定で統計的有意性を確認（ $\chi^2 = 4.65, p = 0.031$ ）。
- Naive RB** は **30** テーブルで完全破綻：7 テーブルでは 96.5% の実行成功率だった Naive RB が、30 テーブルでは全テーブル JOIN により 0% に低下。Schema Graph 走査なしの Rule-based は実運用規模で機能しない。
- No Schema** は実用不可：スキーマ情報なしでは LLM は 0.7% しか正答できず、スキーマ情報注入の決定的重要性を確認。
- 不要 **JOIN** 率の削減：Full Schema 9.2% → Traversed 2.8%。不要 JOIN は正確性（集計水増し）・性能・安全性・保守性の 4 層でリスクとなる（6.1 節で詳述）。

5.10.3. カテゴリ別分析

表 15 にカテゴリ別の Traversal 効果を示す。

Medium カテゴリ（3–4 テーブル結合）での +16pp が最大の改善幅である。これは FK 経路が 1–2 段の間接結合を含む場合に LLM が最も混乱しやすく、Schema Graph 走査による経路明示化の効果が最大となることを示す。Very

Table 13: Graph Traversal アブレーション結果 (30 クエリ ×4 条件、gpt-5.5)。不要 JOIN 数 = max(0, 実際 JOIN 数 - 最小必要 JOIN 数)。

カテゴリ (n)	Naive (all JOINS)		SG + Rule-based		LLM (no traversal)		LLM + SG traversal	
	平均 JOIN	不要 JOIN	平均 JOIN	不要 JOIN	平均 JOIN	不要 JOIN	平均 JOIN	不要 JOIN
0-table (2)	5.00	5.00	0.00	0.00	0.00	0.00	0.00	0.00
1-table (4)	5.00	4.00	1.00	0.00	2.00	1.00	1.00	0.00
2-table (5)	4.80	3.00	1.60	0.00	1.60	0.00	1.80	0.00
3-table (8)	4.75	1.88	2.75	0.00	2.75	0.00	2.75	0.00
indirect (2)	5.00	2.50	0.50	0.00	2.50	0.00	0.50	0.00
multi-element (5)	4.00	1.60	1.60	0.00	1.60	0.00	1.80	0.00
numeric/sort (4)	5.00	3.25	1.75	0.00	1.75	0.00	1.75	0.00
全体 (30)	4.73	2.73	1.67	0.00	1.93	0.13	1.73	0.00

Table 14: 30 テーブル・150 クエリ実験結果 (gpt-4o-mini)。Schema Graph 走査 (Traversed) が全条件で最良の成功率を達成。

条件	成功率	不要 JOIN 率	備考
No Schema	0.7% (1/150)	—	スキーマ情報なし：ほぼ全件失敗
Full Schema (走査なし)	86.7% (130/150)	9.2%	30 テーブル全量注入
Traversed (走査あり)	94.7% (142/150)	2.8%	関連テーブルのみ
Naive RB (全テーブル JOIN)	0% (0/150)	—	30 テーブルで完全破綻
SG+RB (走査あり辞書ベース)	54% (81/150)	—	辞書の限界で半数

Table 15: 30 テーブル・カテゴリ別 Traversal 効果。

カテゴリ	Full	Traversed	改善
Simple	92%	100%	+8pp
Medium	76%	92%	+16pp
Complex	84%	96%	+12pp
Very Complex	96%	96%	0pp
Cross-domain	88%	96%	+8pp
Aggregation	84%	88%	+4pp

Complex カテゴリで改善がないのは、サブクエリ・集約を含むクエリでは JOIN 経路以外の要因（構文の複雑さ）が成功率を支配するためである。

走査なし条件で失敗する 19 件のうち 74% が「スキーマ過多による混乱」（カラム名曖昧化、類似テーブル間の誤参照、FK 経路誤選択）に起因しており、走査あり条件でこれらを正答に転換できた。

6. 考察

6.1. データ組み合わせ精度の重要性：不要 JOIN の多層的リスク

材料工学において、データベース検索の価値は返却行数ではなく異種テーブルの正確な組み合わせにある。組成・構造・安定性・物性が別テーブルに正規化された RDB では、JOIN の正確性がデータ品質の生命線となる。

Graph Traversal アブレーション（表 13）はこの観点を定量的に裏付ける：Naive 条件は平均 4.73 JOIN を生成し 58% が不要である。SG+RB および LLM+SG 条件は FK 走査により不要 JOIN を完全排除し、Jaccard 類似度 0.897 で一致する結果セットを返す。

不要 JOIN がもたらす 4 層のリスク。不要 JOIN は単なる冗長性ではなく、材料データベースにおいて以下の 4 層で実害をもたらす（表 16）。

Table 16: 不要 JOIN の多層的リスク。

リスク層	メカニズム	材料 DB での帰結
正確性	1 対多テーブルの不要結合により行が膨張し、COUNT/AVG 等の集計が水増しされる	格子定数平均値の歪み、補化合物数の過大計上
性能	読み取り行数・メモリ使用量・実行時間が不要テーブルの行数に比例して増大	大規模 DB（100 万件）での応答遅延
安全性	不要テーブル参照によりアクセス制御外のカラム・データが結果に混入する可能性	未公開計算結果の意図しない露出
保守性	SQL が長大化し、意図の把握・レビュー・修正が困難になる	実運用での品質管理コスト増大

具体例：行膨張による集計誤差。「Fe を含む安定な B2 化合物の平均格子定数」を検索する場合を考える。正しい JOIN パスは material_entry→composition→crystal_structure→thermodynamic の 4 テーブルである。ここで不要な band_structure テーブルを追加 JOIN すると、各化合物に対してバンド構造レコード数 (k 点メッシュごとに 1 行、典型的に 3-5 行) 分の重複行が発生する。結果として AVG(lattice_parameter) の計算母数が 3-5 倍に膨張し、個々の値は変わらないが COUNT(*) が過大計上される。さらに深刻なケースとして、composition テーブル（多元素化合物では元素数分の行を持つ）と band_structure の両方を不要 JOIN した場合、行数は「元素数 × バンド構造数」の直積で膨張し、AVG が

意味を持たない値に変質する。

Schema Graph 走査は Steiner 木近似により必要最小限のテーブルのみを JOIN パスとして算出するため、上記 4 層のリスクを原理的に排除する。本実験では走査あり条件で不要 JOIN 率を 2.8% まで削減し（走査なし 9.2%）、7 テーブル環境では完全排除（58%→0%）を達成した。

1,351 件・7 テーブル・5 プロトタイプは本番規模の材料 DB を代表するものではないが、組成・構造・安定性・計算条件を分離した正規化スキーマにおける JOIN パス生成と多元素 AND 検索の主要な故障モードを検証するには十分な複雑性を持つ。重要なのはレコード数ではなく、FK 関係で接続された複数テーブル間の JOIN パス空間であり、7 テーブルの正規化スキーマは材料データベースに典型的な構成である。

オントロジーアプローチとの精度比較. Sequeda & Allemang [23] は汎用ドメイン（保険業界、数十テーブル規模）において GPT-4 zero-shot Text-to-SQL の正解率が 16% に留まり、知識グラフ変換後の Text-to-SPARQL で 54%、さらに OBQC（オントロジーベースクエリ検証）+ LLM 修復で 72% に改善することを報告している。本手法は Schema Graph 制約+ドメイン辞書により 7 テーブル構成で実行成功率 100%・結果正解率 100% を達成した（表 6）。ただし、スキーマ規模（7 テーブル vs 数十テーブル）およびテストセット条件（ドメイン特化キュレーション vs 汎用ベンチマーク）が異なるため直接比較は困難である。本手法の高精度は、(1) FK 走査による最小 JOIN パス保証、(2) 材料用語辞書によるドメイン適応、(3) SQLGuard による事後検証の三重制約に起因する。30 テーブル・150 クエリ検証（5.10 節）では、走査なし条件（Full Schema）の成功率 86.7% に対し、走査あり条件（Traversed）は 94.7%（+8.0pp）に向上し、不要 JOIN 率も 9.2%→2.8% に削減された。特に Medium カテゴリ（3-4 テーブル結合）では走査により +16pp の改善（76%→92%）が確認され、走査なしでは 19 件が失敗するクエリを走査ありで正答に転換できた。

6.2. 多次元評価の統合的考察

30 テーブル・150 クエリ検証（表 14）と Core サブセットアブレーション（表 6）の結果を統合すると、以下の設計原則が導出される：

- スキーマ情報注入が決定的：7 テーブルで条件 2（LLM-only, 1.8%）vs 条件 6（SG+LLM, 100%）、30 テーブルで No Schema（0.7%）vs Traversed（94.7%）の差異が示す。
- Schema Graph 走査はスキーマ規模に比例して重要性が増大：Core サブセット（7 テーブル、星型構造）では LLM が JOIN パスを「記憶」可能で天井効果が生じるが、30 テーブルでは走査なし 86.7% → 走査あり 94.7%（+8.0pp）と明確な差異が出現。Naive RB は 30 テーブルで 0% に破綻。
- RAG 効果はモデル能力に依存：gpt-5.5 では RAG の追加効果が消失するが、gpt-4o-mini では 80%→98%（+18pp）の顕著な改善が確認された。
- Rule-based モードの実用性：Core サブセットでは外部 API 依存なしで 100% を達成し、30 テーブルでも SG+RB で 54%（辞書カバー範囲内）を達成。

6.3. LLM の能力向上は Schema Graph 走査を不要にするか

「十分に高性能な LLM であれば Schema Graph 走査は不要になるのではないか」という自然な疑問に対し、本実験結果は明確に否定的である。

実験的証拠：最先端モデルでも精度低下. gpt-5.5 は 2026 年時点での最高性能モデルの一つであり、7 テーブルでは天井効果（100%）を示す。しかしスキーマ複雑性が増加した 30 テーブル環境では、gpt-4o-mini による検証で走査なし条件が 86.7% に低下することが確認された。表 17 に 2 モデルでの比較を示す。

Table 17: モデル世代別の Traversal 効果。天井効果はモデル能力の高さに起因するが、テーブル数増加とともに走査の価値が顕在化する。

モデル	7 テーブル	30 テーブル (Full)	30 テーブル (Tr)
gpt-5.5	100%	—	—
gpt-4o-mini	80% (RAG なし)	86.7%	94.7%

両モデルでの検証により、**Schema Graph** 走査がモデル世代に依存しない構造的改善であることが示される—モデル能力が低いほど効果が大きく、高性能モデルでも JOIN 最適性は改善される。

組み合わせ爆発：LLM コンテキストの本質的限界. 30 テーブル・31FK 関係のスキーマでは、2 テーブル間の JOIN パス候補は理論上 $O(n!)$ の経路を含み得る。本実験の 30 テーブルスキーマでは約 3,000 トークンのスキーマ記述となり、LLM の注意機構がすべての FK 関係を正確に追跡することは保証されない。実際に、Full Schema 条件での失敗 19 件のうち 74% が「スキーマ過多による混乱」（カラム名曖昧化、類似テーブル間の誤参照）に起因する。Schema Graph 走査は関連テーブルのみ（平均 3-5 テーブル、~500 トークン）を LLM に提示するため、この組み合わせ爆発を回避する。

構造的保証 vs 確率的推論. Schema Graph 走査が提供する保証は LLM では代替不可能である：

- 最小性：Steiner 木近似は証明可能な最小 JOIN パスを返す。LLM は「おそらく正しい」パスを返すが、最小性の保証はない。
- 決定論性：同一クエリに対して常に同一の JOIN パスが選択される。gpt-5.5 の SQL 一致率は 30% であり、材料研究の再現性要件と相容れない。
- 説明可能性：FK 走査の結果はグラフ上の経路として可視化・検証可能である。LLM の内部推論過程はブラックボックスであり、JOIN パスの正当性を第三者が検証できない。
- スケーラビリティ：Steiner 木近似は多項式時間で計算可能であり、50-100 テーブル規模でも一定の品質保証を提供する。LLM のコンテキストウィンドウはスキーマ規模に対してサブリニアにスケールする。

結論として、LLM は SQL 構文の生成を改善するが、JOIN 経路の正確性は構造的問題であり、確率的モデルでは原理的に保証不能である。Schema Graph 走査は LLM の補完ではなく、LLM が解決できない構造的保証を提供する不可欠な構成要素である。

Table 18: 主要 3 モードの利点と欠点。

モード	利点	欠点
Naive RB	最小実装(約 50 行)。依存なし。最速 (32 ms)	不要 JOIN。多元素 AND 失敗。安全性なし。LIMIT/DISTINCT なし
SG + RB	外部 API 不要。32-60 ms。完全な安全性。100%決定論的。Coverage Score で未認識語彙検出	辞書外語彙で LLM フォールバック必要。否定/複雑条件不可
SG + LLM	未登録語彙対応。数値/否定可能。文脈理解。100%実行成功 (gpt-5.5)	外部 API 依存 (2.4-6.1 秒、コスト)。非決定論的 (SQL 一致率 30%)。Out-of-scope 質問に SQL 生成 (Intent Classifier 必要)

6.4. 各モードの利点と欠点

6.5. 境界と制約

6.5.1. テストケースの自己参照性 (深刻度: 高)

80 件の回帰テストは抽出パターンを知るシステム開発者が設計した。したがって、制御された条件下で 100% のパス率は予想される。100 件の VASP ストレステストは半独立的に設計されたが、真の精度評価には材料研究者による自由文クエリのブラインドテストが必要である。今後の課題として、第三者の材料研究者複数名によるブラインドミニセット (例: 5 名 × 5 クエリ) を用いた独立評価を計画している。

6.5.2. RAG 天井効果 (深刻度: 中 → 解決済み)

RAG アブレーション (4 条件 × 50 クエリ) は gpt-5.5 で天井効果を示し、Few-Shot 事例の追加効果が測定不能であった。追実験として gpt-4o-mini (小型モデル) で同一 50 クエリを再実行したところ、No Examples (Schema Graph プロンプトのみ) 80%、Manual Only 82%、Paper Only 84%、All Examples 98% と明確な RAG 効果が確認された (表 11)。これは Schema Graph 制約がベースラインの品質を保証しつつ、RAG が小型モデルの能力不足を補完する設計であることを実証する。gpt-5.5 での天井効果はモデル能力が Schema Graph 制約で飽和したためであり、RAG システム自体の無効性を意味しない。

6.5.3. 無通知失敗 (深刻度: 中 → 緩和済み)

Rule-based モードでの未登録語彙の無通知破棄 (Silent Constraint Dropping) は、Coverage Score の実装により緩和された。スコア 0.8 未満で LLM フォールバック、0.5 未満で明確化要求を発行する。ただし、Coverage Score の閾値は経験的に設定されており、最適化の余地がある。

6.5.4. Out-of-scope 検出 (深刻度: 中 → 部分解決)

VASP ストレステストで out-of-scope 正解率 0% が判明した。Intent Classifier (3.2.7 節) を実装し 20 パターンの

VASP ワークフロー検出を追加したことで out-of-scope 正解率は 100% に改善した (表 9) が、正規表現ベースのため、新しいパターンへの汎化は限定的である。LLM ベースの Intent 分類への拡張が今後の課題である。

6.5.5. スキーマ規模の妥当性 (深刻度: 中 → 低)

本研究は OQMD 実規模を想定した 30 テーブル・31FK 関係スキーマで主検証を実施し (5.10 節)、走査なし 86.7% → 走査あり 94.7% (+8.0pp) の改善を実証した。Core サブセット (7 テーブル・1,351 レコード) でのアブレーション実験も実施している。

表 19 に 30 テーブル検証で観察された失敗パターンと Schema Graph 走査による解決機序を示す。

Table 19: 30 テーブル検証における失敗パターンと Schema Graph 走査による緩和。

失敗パターン	Full Schema 注入時の原因	Traversal による緩和
カラム名曖昧参照	複数テーブルに同名カラム (entry_id 等) が存在し、LLM が誤ったテーブルから参照	関連テーブルのみ提示することで候補を限定
存在しないカラム参照	類似テーブル・類似カラム名から LLM が幻覚的に生成	スキーマコンテキスト縮小により幻覚を抑制
FK 経路の誤選択	直接 JOIN を試行するが中間テーブルが必要	FK 走査が Steiner 木で正しい経路を明示
不要 JOIN	提示された 30 テーブルから不要なテーブルまで JOIN	最小サブグラフのみ提示 不要 JOIN を原理的に排除

6.5.6. LLM 再現性 (深刻度: 低)

gpt-5.5 の SQL 一致率は 30% (20 クエリ × 5 回) であり、同一クエリに対して構造的に異なる SQL が生成される。ただし実行成功率は 100% であり、意味的等価性は保持されている。テスト自動化では結果セット比較が推奨される。

6.6. 関連システムとの比較

6.7. 手法能力比較

表 21 に、本手法と既存アプローチの材料データベース向け主要能力を比較する。

6.8. AI for Science (AI4S) における意義

本手法は AI4S パラダイムに直接的な含意を持つ。表 22 に AI4S ワークフローの各段階と T2SQL 手法の役割を示す。

重要な点として、Schema Graph アーキテクチャはスキーマ非依存である: FK 関係は実行時に PostgreSQL メタデータからイントロスペクション (3.3 節) されるため、OQMD に限らず任意の正規化 RDB (Materials Project、AFLOW、NOMAD、機関内データベース、さらには材料科学以外のドメイン) にコード変更なしで展開できる—用語辞書のドメイン適応のみが必要である。とりわけ、Materials Integration (MIInt) システム [31] における PSPP (プロセス・構造・物性・性能) 連鎖の各モジュールが出力する RDB や、provenance

Table 20: 関連する材料 NLP / T2SQL / オントロジーシステムとの比較。

システム	スキーマ認識	正規化 RDB	材料特化	NL→ クエリ	主な差異
Spider [11]	あり	あり	なし	SQL	汎用ベンチマーク
BIRD [12]	あり	あり	なし	SQL	大規模実データ
DAIL-SQL [13]	あり	あり	なし	SQL	スケルトン選択
MKNA [19]	なし	なし	あり	API	エージェント
OptiMat [20]	なし	なし	あり	API	オンデマンド DFT
SuperCon [9]	あり	なし	あり	SPARQL	RDF オントロジー
PoLyInfo [10]	あり	なし	あり	SPARQL	BFO 準拠
EMMO [22]	あり	なし	あり	—	上位オントロジー
KG+OBQC [24]	あり	なし	なし	SPARQL	72%精度
本手法	あり	あり	あり	SQL	FK 走査 + 材料辞書

Table 21: 材料 Text-to-SQL における手法能力比較。

手法	ドメイン辞書	FK 走査	SQL ガード	材料パーサー	正規化組成対応
汎用 LLM T2SQL (zero-shot)	—	—	部分的	—	—
Few-shot T2SQL	—	—	部分的	—	—
KG / Text-to-SPARQL	オントロジー	グラフ走査	あり	ドメイン依存	RDF 経由
DAIL-SQL [13]	—	—	—	—	—
OBQC+LLM [24]	オントロジー	グラフ走査	あり	—	—
本手法	あり	あり	8 層	あり	あり

Table 22: AI4S 材料探索ワークフローにおける T2SQL の役割。

AI4S 段階	従来アプローチ	T2SQL 手法適用時
仮説生成	研究者が手動でクエリを作成	LLM エージェントが仮説から NL クエリを生成
データ検索	SQL または API 呼び出しを手動記述	NL → SQL 自動変換 (Schema Graph 経由)
スクリーニング	反復的 SQL 修正	Few-Shot RAG が成功クエリパターンを再利用
結果解釈	クエリ結果の手動検査	メタデータ付き構造化出力 (プロトタイプ、安定性、物性)
フィードバック	知識は個々の研究者に留まる	成功クエリが Few-Shot ストアに蓄積され再利用

管理システム pinax [32] が蓄積する解析ワークフローデータベースは、本手法の自然な適用先として期待される。本研究で OQMD を検証対象に選定した理由は、組成・構造・熱力学的安定性の外部キー接続による 7 テーブル構成が材料データベースに典型的なスキーマ複雑性の好例であり、ここでの成功が他の RDB への展開可能性を強く示唆するためである。

さらに、カスケード Rule-based → LLM アーキテクチャ (3.7 節) は、AI4S が要求する信頼性とコスト効率の要件に適合する：定型クエリは外部 API 呼び出しなしに決定論的に処理され、複雑なクエリのみが LLM 能力を活用する。このハイブリッドアプローチにより、AI4S パイプラインは禁止的な API コストやレイテンシなしに大規模運用が可能になる。

6.9. 今後の展望

本研究で実装済みの項目（数値条件パーサー、Coverage Score、RAG アブレーション、Intent Classifier）を踏まえ、残る課題を示す：

1. マルチデータベース拡張：Schema Graph を Materials Project、AFLOW スキーマに適応。動的 FK イントロスペクションはアーキテクチャ上これを既にサポートしている。
2. **AI4S** エージェント統合：T2SQL 手法を自律材料探索エージェントのツールとして組み込み（MCP プロトコルや function calling 経由）、仮説 → データ検索 → 分析のエンドツーエンドパイプラインを実現。
3. ブラインドユーザースタディ：10 名以上の材料研究者から自由文クエリを収集し、テストケースの自己参照性問題を解消する。
4. 高難度クエリセットでの **RAG** 再評価：複雑な集約・ウィンドウ関数・多テーブル結合クエリで RAG 条件間差異を検証し、天井効果の範囲を明確化する。
5. **LLM** ベース **Intent** 分類：正規表現ベースの Intent Classifier を軽量 LLM（ローカル推論可能なモデル）に置換し、新しいスコープ外パターンへの汎化性能を向上する。
6. 大規模レコード検証：30 テーブル・150 クエリの検証は実施済みだが、10 万レコード以上での Steiner 木近似の計算性能およびクエリ実行性能の検証が残る。
7. **CALPHAD** 連携による熱力学的精度向上：Text-to-SQL で取得した候補化合物のうち、熱力学的に特定が困難なもの（準安定相、固溶体の安定性判定等）については、CALPHAD 法（Thermo-Calc 等の TDB ファイルからギブズエネルギー関数・相互作用パラメータを抽出）と連携することで精度向上が可能である。具体的には、Schema Graph 走査で RDB から候補を絞り込んだ後、CALPHAD の自由エネルギー計算により相安定性を判定する二段階フローを想定する。これにより、RDB 上の E_{hull} 近似では判定困難な温度依存相安定性や多成分固溶体の分解挙動にも対応可能となる。
8. クロスリンガル埋め込み検索：TF-IDF を transformer 埋め込みに置換し Few-Shot 類似度を改善—国際 AI4S 協力において多言語クエリが到着する前提条件。
9. 対話的クエリ精緻化：曖昧クエリに対する段階的確認質問の返却（VASP ストレステストで曖昧カテゴリ 40% が課題として残る）。

7. 結論

本研究では、正規化された材料リレーショナルデータベースに対する Schema Graph 制約型自然言語インターフェースを提案した。材料データベースにおける自然言語アクセスの本質的課題は **SQL** 構文の生成ではなく、正規化された異種材料データを外部キー関係と材料ドメイン意味論に従って正しく結合することである—本手法はこの課題に Schema Graph 走査（Steiner 木近似）で対応する。

OQMD 実規模を想定した **30 テーブル・31FK 関係・150 クエリ** の検証を主軸に、Core サブセット（7 テーブル・1,351 件）でのアブレーション実験を含め本手法を評価した。主要な知見：

1. **Schema Graph** 走査は実運用規模で不可欠：30 テーブル環境で走査なし 86.7% → 走査あり 94.7% (+8.0pp)。不要 JOIN 率 9.2% → 2.8%。Medium カテゴリでは +16pp 改善。Naive RB（全テーブル JOIN）は 30 テーブルで実行成功率 0% に破綻。
2. スキーマ情報注入が決定的：7 条件比較（gpt-5.5）により、Schema Graph 制約を含む全条件が実行成功率 100% を達成。LLM-only（スキーマ情報なし）は 1.8%。
3. **Rule-based** モードの実用性：外部 API 依存ゼロで 32–60 ms のレイテンシにて 57/57 テスト成功。辞書カバー範囲内クエリには LLM 呼び出しが不要。
4. **RAG** 効果はモデル能力に依存：gpt-5.5 では天井効果（全条件 100%）だが、gpt-4o-mini では 80% → 98% (+18pp)。
5. **Intent Classifier**：VASP ワークフロー質問の検出により、out-of-scope 正解率を 0% から 100% に改善。
6. **8 層 SQL** ガードは全インジェクション攻撃を遮断し、結果セット上限を強制する。

実証の限界：30 テーブルスキーマは OQMD 実規模を想定した設計だがシミュレーションデータでの検証に限定される。クエリセットは著者設計であり、独立したユーザー評価は今後の課題である。

カスケード Rule-based → LLM アーキテクチャは速度・コスト・カバレッジのバランスを取り、材料研究者が SQL 専門知識なしに正規化データベースを自然言語でクエリすることを可能にする。

AI for Science (AI4S) のより広い文脈において、本研究は正規化 RDB への自然言語アクセスという AI 駆動型科学ワークフローの基盤的課題に取り組む。Schema Graph T2SQL 手法は、実行時 FK イントロスペクションに基づくため、Materials Project、AFLOW、機関内データベース等への移植が可能な設計である。レコード数 10 万以上の大規模データおよび材料科学以外のドメインでの有効性は今後の検証課題である。

現在の検証は OQMD 単一データベースに限定されるが、本手法は正規化 RDB 全般に対する T2SQL の再現可能なフレームワークを確立し、マルチデータベース検証、大規模スキーマ対応、AI4S エージェントアーキテクチャへの統合へと拡張可能である。

謝辞

DFT 計算データは Open Quantum Materials Database (OQMD) から取得した。OQMD の開発・維持管理にあた

る Northwestern 大学 Wolverton グループに感謝する。本研究は NIMS の支援を受けた。

References

- [1] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac, A. Anandkumar, K. Bergen, C. P. Gomes, S. Ho, P. Kohli, J. Lasenby, J. Leskovec, T.-Y. Liu, A. Manber, D. Misra, E. Moret, J. Pineau, B. Poole, M. Sacks, S. SenGupta, J. Sun, J. Tang, A. Trischler, M. Welling, Y. Xie, M. Zitnik, Scientific discovery in the age of artificial intelligence, *Nature* 620 (2023) 47–60. doi:10.1038/s41586-023-06221-2.
- [2] J. E. Saal, S. Kirklin, M. Aykol, B. Meredig, C. Wolverton, Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD), *JOM* 65 (2013) 1501–1509. doi:10.1007/s11837-013-0755-4.
- [3] S. Kirklin, J. E. Saal, B. Meredig, A. Thompson, J. W. Doak, M. Aykol, S. Rühl, C. Wolverton, The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies, *npj Comput. Mater.* 1 (2015) 15010. doi:10.1038/npjcompumats.2015.10.
- [4] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, K. A. Persson, Commentary: The Materials Project: A materials genome approach to accelerating materials innovation, *APL Mater.* 1 (2013) 011002. doi:10.1063/1.4812323.
- [5] S. Curtarolo, W. Setyawan, G. L. W. Hart, M. Jahnatek, R. V. Chepulskii, R. H. Taylor, S. Wang, J. Xue, K. Yang, O. Levy, M. J. Mehl, H. T. Stokes, D. O. Demchenko, D. Morgan, AFLOW: An automatic framework for high-throughput materials discovery, *Comput. Mater. Sci.* 58 (2012) 218–226. doi:10.1016/j.commatsci.2012.02.005.
- [6] C. Draxl, M. Scheffler, NOMAD: The FAIR concept for big data-driven materials science, *MRS Bull.* 43 (2018) 676–682. doi:10.1557/mrs.2018.208.
- [7] O. N. Senkov, D. B. Miracle, K. J. Chaput, J.-P. Couzinie, Development and exploration of refractory high entropy alloys — A review, *J. Mater. Res.* 33 (2018) 3092–3128. doi:10.1557/jmr.2018.153.
- [8] E. P. George, D. Raabe, R. O. Ritchie, High-entropy alloys, *Nat. Rev. Mater.* 4 (2019) 515–534. doi:10.1038/s41578-019-0121-4.
- [9] M. Ishii, K. Sakamoto, Structuring superconductor data with ontology: Reproducing historical datasets as knowledge bases, *Science and Technology of Advanced Materials: Methods* 3 (1) (2023) 2223051. doi:10.1080/27660400.2023.2223051.
- [10] M. Ishii, T. Ito, K. Sakamoto, NIMS polymer database PoLy-Info (II): machine-readable standardization of polymer knowledge expression, *Science and Technology of Advanced Materials: Methods* 4 (1) (2024) 2354651. doi:10.1080/27660400.2024.2354651.
- [11] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: *Proceedings of EMNLP*, 2018, pp. 3911–3921.
- [12] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., Can LLM already serve as a database interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQL (BIRD), *Advances in Neural Information Processing Systems* 36 (2024).
- [13] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, *arXiv preprint arXiv:2308.15363* (2023).
- [14] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2024.
- [15] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, X. Huang, Next-generation database interfaces: A survey of LLM-based text-to-SQL, *arXiv preprint arXiv:2406.08426* (2024).
- [16] K. Maamari, F. Abubaker, D. Jaroslawicz, A. Mhedhbi, The death of schema linking? text-to-SQL in the age of well-reasoned language models, *arXiv preprint arXiv:2408.07702* (2024).
- [17] Y. Song, S. Miret, B. Liu, MatSci-NLP: Evaluating scientific language models on materials science language tasks using text-to-schema modeling, in: *Proceedings of the 61st Annual Meeting of the ACL*, 2023, pp. 3621–3639.
- [18] V. Mishra, S. Singh, M. Zaki, et al., LLAMAT: Large language models for materials science information extraction, *arXiv preprint arXiv:2411.00177* (2024).
- [19] G. Zhuang, A. B. Farimani, From natural language to materials discovery: The Materials Knowledge Navigation Agent, *arXiv preprint arXiv:2602.11123* (2026).
- [20] Y. Hu, V. Turlo, OptiMat Alloys: A FAIR end-to-end agent with living database for computational multi-principal alloy exploration, *arXiv preprint arXiv:2604.21850* (2026).
- [21] B. Debrah, *Materials Project MCP Server*, model Context Protocol integration for Materials Project database (2025). URL <https://www.pulsemcp.com/servers/benedictdebrah-materials-project>
- [22] EMMC-CSA Consortium, *Elementary multiperspective material ontology (EMMO)*, version 1.0.0, CC BY 4.0 (2020). URL <https://github.com/emmo-repo/EMMO>
- [23] J. F. Sequeda, D. Allemang, B. Jacob, A benchmark to understand the role of knowledge graphs on large language model's accuracy for question answering on enterprise SQL databases, in: *Proceedings of the 7th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, ACM, 2024. doi:10.1145/3661304.3661901.
- [24] D. Allemang, J. F. Sequeda, Increasing the LLM accuracy for question answering: Ontologies to the rescue!, *arXiv preprint arXiv:2405.11706* (2024).
- [25] A. A. Hagberg, D. A. Schult, P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, *proceedings of the 7th Python in Science Conference (SciPy)* (2008).
- [26] F. K. Hwang, D. S. Richards, P. Winter, *The Steiner Tree Problem*, North-Holland, 1992.
- [27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, *Advances in Neural Information Processing Systems* 33 (2020) 9459–9474.
- [28] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, *Inf. Process. Manage.* 24 (1988) 513–523. doi:10.1016/0306-4573(88)90021-0.
- [29] T. Mao, SQLGlot: A python SQL parser, transpiler, optimizer, and engine, <https://github.com/tobymao/sqlglot> (2023).
- [30] P. Jaccard, The distribution of the flora in the alpine zone, *New Phytologist* 11 (2) (1912) 37–50. doi:10.1111/j.1469-8137.1912.tb05611.x.
- [31] S. Minamoto, T. Kadohira, K. Ito, M. Watanabe, Development of the Materials Integration system for materials design and manufacturing, *Materials Transactions* 61 (11) (2020) 2067–2071. doi:10.2320/matertrans.MT-MA2020002.
- [32] S. Minamoto, T. Kadohira, J. Fujima, Y. Fujiwara, A. Endo, C. Suematsu, K. Daimaru, H. Izuno, J. Sakurai, M. Demura, pinax: a provenance management system for materials data science, *Science and Technology of Advanced Materials: Methods* 6 (1) (2026) 2629051. doi:10.1080/27660400.2026.2629051.

A. 回帰テストケース一覧

本付録には回帰テスト 44 件 (表 23) と代表的 SQL 生成例を掲載する。全評価クエリ (57 件 Baseline 比較、50 件 RAG アブレーション、30 件 Graph Traversal アブレーション、100 件 VASP ストレステスト、20 件 LLM 再現性テスト) の完全リスト、プロンプトテンプレート、allowed_schema.yaml、material_terms.yaml は Supplementary Materials に収録する。

```
Schema Graph + Rule-based (Level 1) : .「  
SELECT DISTINCT m.entry_id, m.formula  
FROM material_entry m  
WHERE EXISTS (  
  SELECT 1 FROM composition  
  WHERE entry_id = m.entry_id  
  AND element = 'Ni')  
AND EXISTS (  
  SELECT 1 FROM composition  
  WHERE entry_id = m.entry_id  
  AND element = 'Al')  
LIMIT 100;  
-- EXISTS副問い合わせで正確な多元素AND
```

B. 生成 SQL 例

B.1. A01: 「Fe を含む B2 化合物を出して」

```
Naive (Level 0) : .「  
SELECT m.entry_id, m.formula,  
  s.prototype  
FROM material_entry m  
JOIN composition c  
  ON c.entry_id = m.entry_id  
JOIN structure s  
  ON s.entry_id = m.entry_id  
JOIN phase_stability ps  
  ON ps.entry_id = m.entry_id  
JOIN calculation calc  
  ON calc.entry_id = m.entry_id  
JOIN calculated_property cp  
  ON cp.calculation_id  
  = calc.calculation_id  
WHERE s.prototype = 'B2'  
AND c.element = 'Fe';  
-- 問題: 不要JOIN 3件, LIMIT/DISTINCTなし
```

```
Schema Graph + Rule-based (Level 1) : .「  
SELECT DISTINCT m.entry_id, m.formula,  
  s.prototype, s.lattice_a, s.space_group  
FROM material_entry m  
JOIN composition c  
  ON c.entry_id = m.entry_id  
JOIN structure s  
  ON s.entry_id = m.entry_id  
WHERE (s.prototype = 'B2'  
OR s.strukturbericht = 'B2')  
AND c.element = 'Fe'  
LIMIT 100;  
-- 最適: 必要な2 JOINのみ、DISTINCT、LIMIT付き
```

```
Schema Graph + gpt-5.5 (Level 2) : .「  
SELECT DISTINCT m.entry_id, m.formula,  
  s.prototype, s.lattice_a, s.space_group  
FROM material_entry m  
JOIN structure s  
  ON s.entry_id = m.entry_id  
JOIN composition c  
  ON c.entry_id = m.entry_id  
WHERE (s.prototype = 'B2'  
OR s.strukturbericht = 'B2')  
AND c.element = 'Fe'  
LIMIT 100;  
-- LLMがSG制約内で同等SQLを生成
```

B.2. A03: 「Ni と Al の両方を含む化合物を出して」

```
Naive (Level 0) : .「  
SELECT m.entry_id, m.formula  
FROM material_entry m  
JOIN composition c  
  ON c.entry_id = m.entry_id  
...(全テーブルJOIN)...  
WHERE c.element = 'Ni'  
AND c.element = 'Al';  
-- 致命的: 単一行が両条件を同時に  
-- 満たすことは不可能 -> 常に0行
```

Table 23: 回帰テストケース一覧（全 44 件、全 PASS）。

ID	カテゴリ	自然言語クエリ	結果
A01	正常系	Fe を含む B2 化合物を出して	PASS
A02	正常系	安定な L1 ₂ 化合物を形成エネルギー順に出して	PASS
A03	正常系	Ni と Al の両方を含む化合物を出して	PASS
A04	正常系	B2 化合物の全リストを出して	PASS
A05	正常系	準安定な B2 化合物を出して	PASS
A06	正常系	Co を含む安定な L1 ₂ 化合物を出して	PASS
A07	正常系	Fe と Al を含む B2 と L1 ₂ 化合物を出して	PASS
A08	正常系	γ' 化合物を出して	PASS
A09	正常系	ニッケルを含む L1 ₂ 型化合物を出して	PASS
A10	正常系	L1 ₂ 化合物の全データを出して	PASS
A11	正常系	Ti を含む B2 化合物を格子定数順（降順）に出して	PASS
A12	正常系	CsCl 型化合物を出して	PASS
A13	正常系	Cu ₃ Au 型化合物を出して	PASS
A14	正常系	鉄を含む B2 化合物を出して	PASS
A15	正常系	安定な L1 ₂ 化合物で Ni を含むものを出して	PASS
B01	該当なし	Xe を含む B2 化合物を出して	PASS
B02	該当なし	U と Pu を含む L1 ₂ 化合物を出して	PASS
B03	該当なし	Rn と Og を含む B2 化合物を出して	PASS
B04	該当なし	Sc と Ir を含む安定な B2 化合物を出して	PASS
B05	該当なし	Pt と Ga を含む安定な B2 化合物を出して	PASS
C01	曖昧	安定な化合物を出して	PASS
C02	曖昧	B2	PASS
C03	曖昧	安定なものを出して	PASS
C04	曖昧	（空入力）	PASS
C05	曖昧	今日の天気を教えて	PASS
C06	曖昧	Fe	PASS
C07	曖昧	格子定数	PASS
C08	曖昧	band gap が大きい安定な B2 化合物	PASS
C09	曖昧	NiAl L12	PASS
C10	曖昧	Fe の B2 か L12	PASS
D01	矛盾	安定かつ準安定な B2 化合物	PASS
D02	矛盾	Fe なしの Fe B2 化合物	PASS
E01	拒否	DROP TABLE material_entry;	PASS
E02	拒否	SELECT *; DELETE FROM composition;	PASS
E03	拒否	B2 化合物; DROP TABLE structure;	PASS
E04	拒否	SELECT * FROM secret_passwords	PASS
E05	拒否	INSERT INTO material_entry VALUES (...)	PASS
F01	安全性	SELECT entry_id FROM material_entry	PASS
F02	安全性	UPDATE material_entry SET formula='X'	PASS